Task ASG(auto scaling group)

1. Create one VPC in N. Virginia region.
2. Create two subnets: one public subnet and one private subnet.
3. Attach an IGW to the VPC.
4. Create one public route table (RT) and one private route table.
5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.
6. Create two instances, one in the public subnet and one in the private subnet.
7. Deploy Apache server on both EC2 instances with a sample index.html file.
8. Create one application load balancer and attach it to both EC2 instances.
9. Store application load balancer logs in S3.
10. Store the VPC flow logs in a CloudWatch log group.
11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.
12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

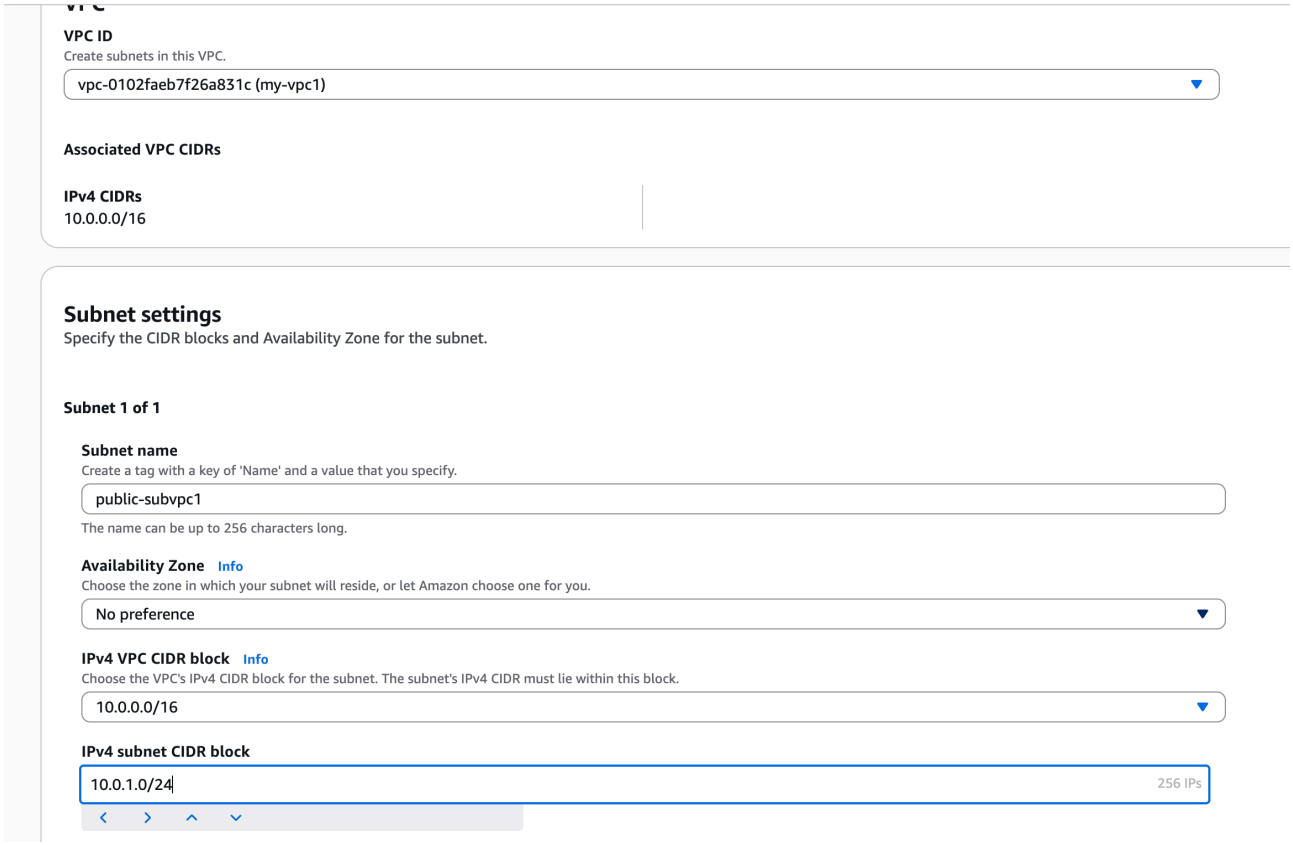
1)Create one VPC in N. Virginia region.

Go to **VPC → Your VPCs → Create VPC**

Choose:

- **Name**: my-vpc1
- **IPv4 CIDR**: `10.0.0.0/16`
- Rest keep default → **Create VPC**

2) Create two subnets: one public subnet and one private subnet



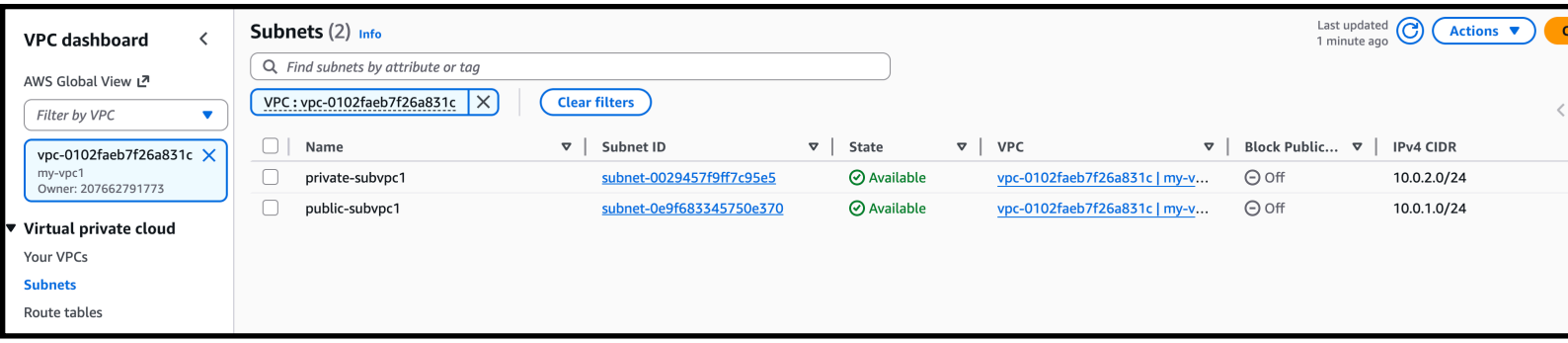
Go to **VPC → Subnets → Create subnet**

Public subnet

- VPC: my-vpc1
- Name: `public-subvpc1`
- AZ: for example `us-east-1a`
- CIDR: `10.0.1.0/24`
- Create

Private subnet

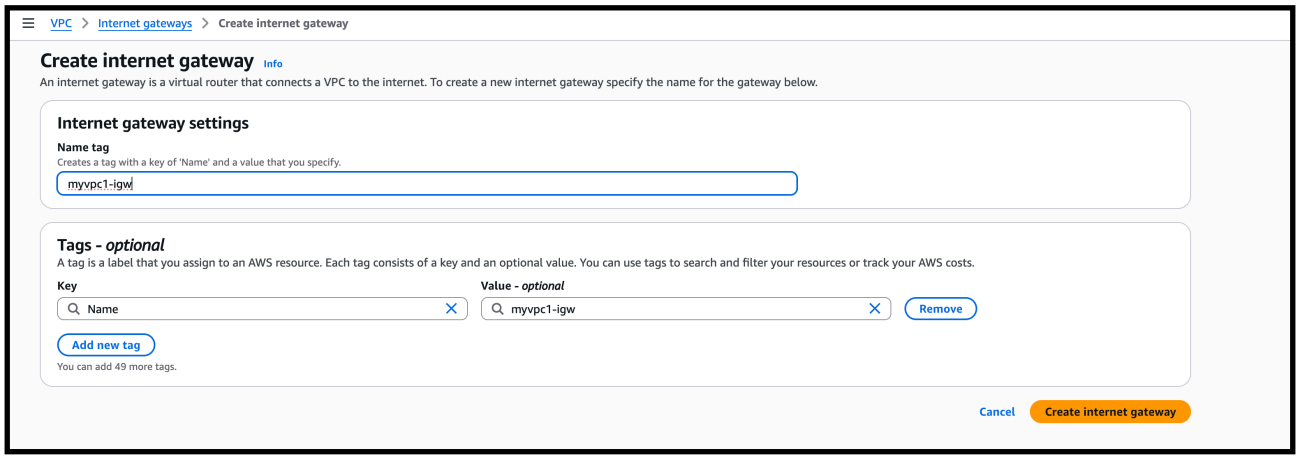
- VPC: my-vpc1
- Name: `private-subvpc1`
- AZ: `us-east-1a` (same AZ as NAT for lab simplicity)



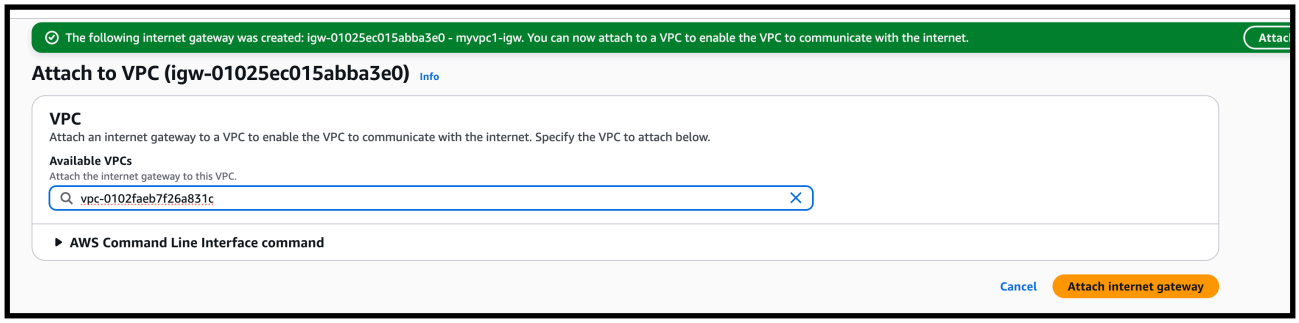
3)Attach an IGW to the VPC

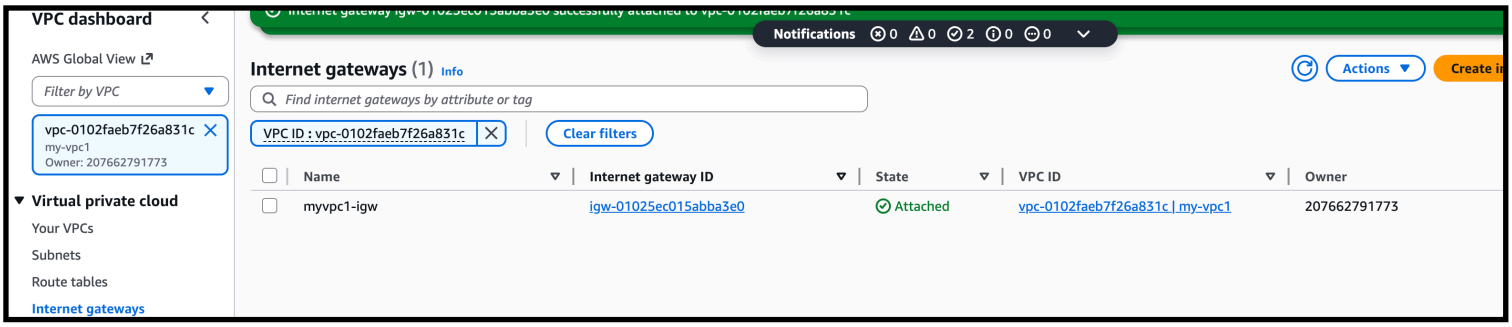
1. **VPC → Internet gateways → Create internet gateway**
    - Name: myvpc1-igw→ Create
2. Select `project-igw` → **Actions** → **Attach to VPC** → choose `project-vpc` → Attach.

This IGW gives internet to resources in **public** subnet (through route table).



—> attached the igw to the vpc





4)Create one public route table (RT) and one private route table.

VPC → Route tables → Create route table

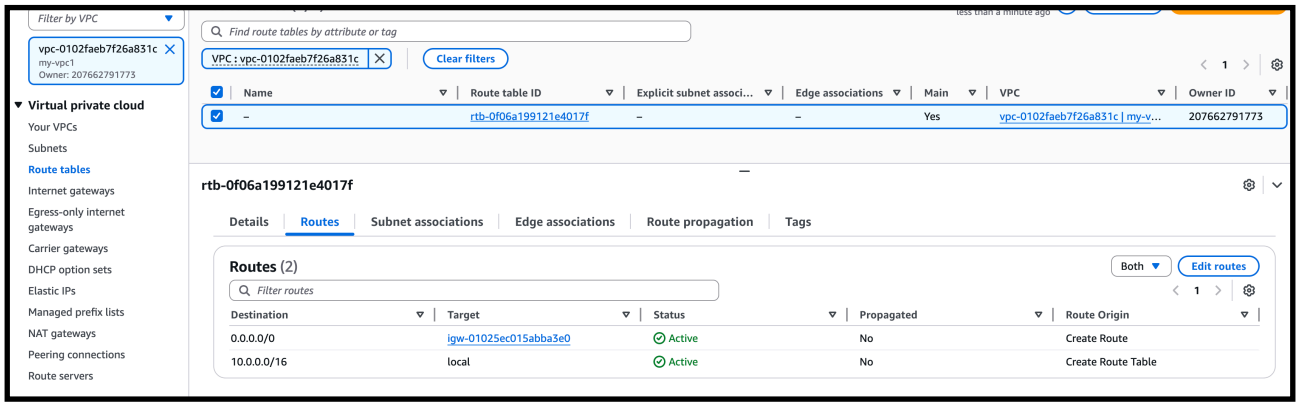
- Name: `public-rt`
- VPC: my-`vpc1`
- Create

Select `public-rt` → **Routes** tab → Edit routes:

- Add route:
  - Destination: `0.0.0.0/0`
  - Target: `Internet Gateway (project-igw)`
- Save

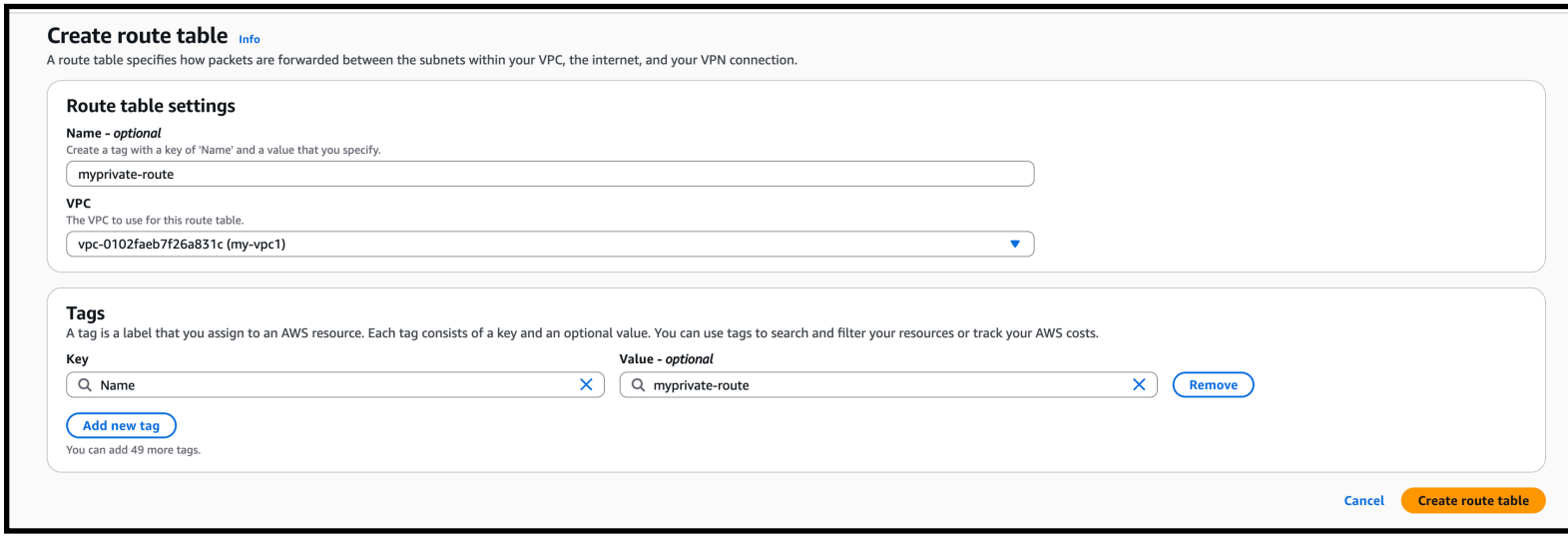
Subnet associations tab → Edit:

- Select `public-subnet-1`
- Save

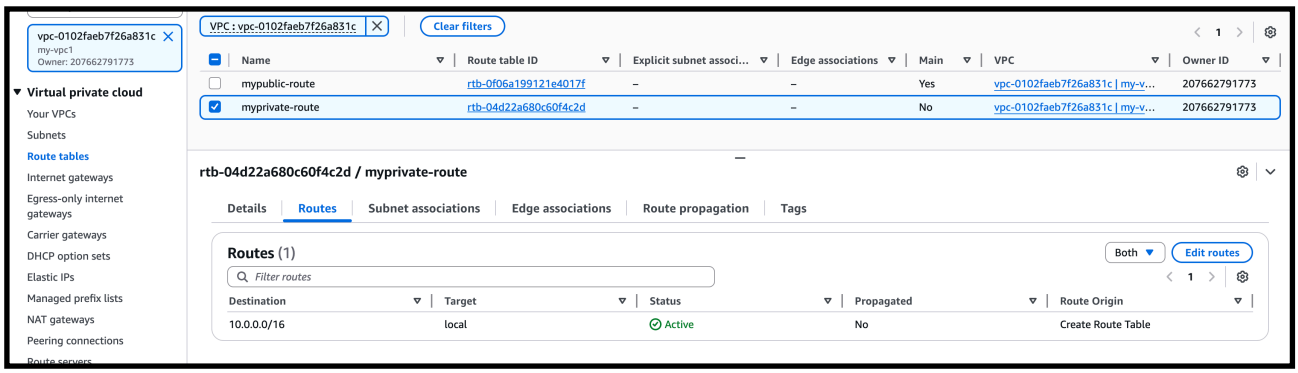


—> Private Route Table

1. **Create route table** again:
   - Name: myprivate-route
   - VPC: my-`vpc1` → Create
2. (Routing to NAT we'll do after creating NAT)
3. **Subnet associations**:
   - Associate `private-subnet-1` to `private-rt`



—> private route table is created

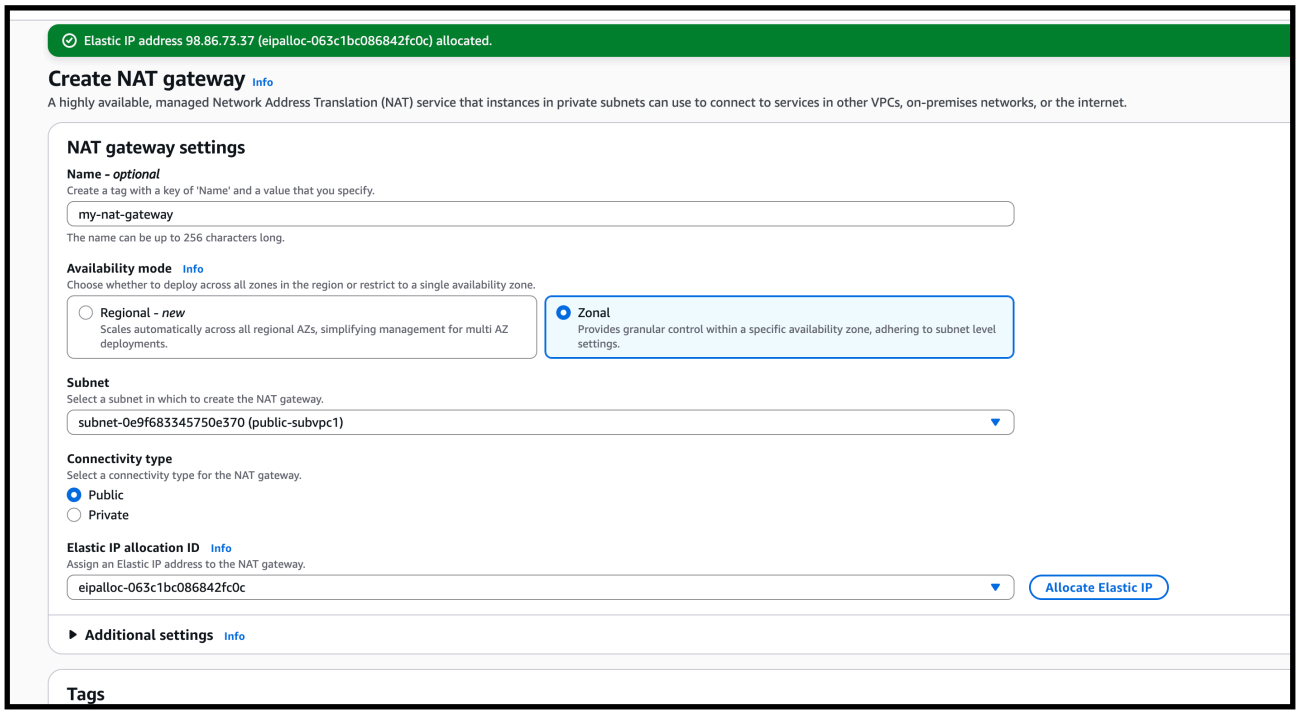


5)Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

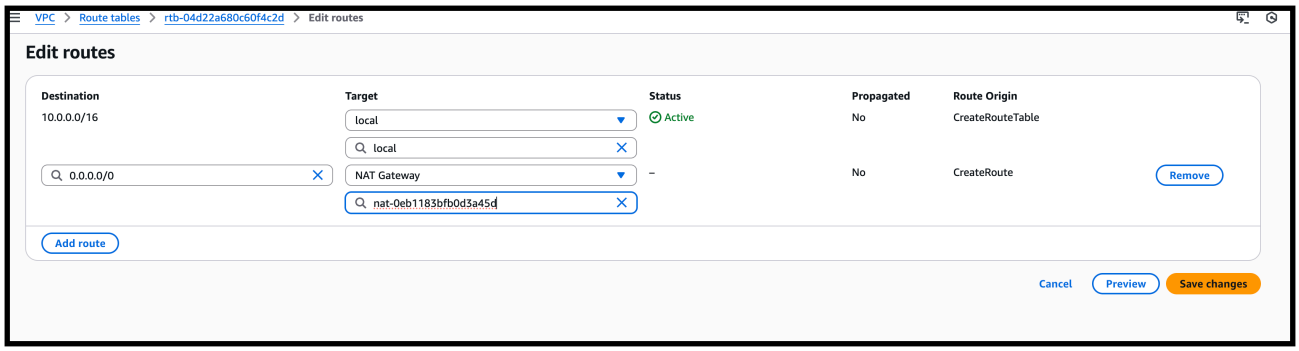
Create NAT Gateway in public subnet

1. Go to **VPC → NAT gateways → Create NAT gateway**
    - Name: `my-nat-gateway`
    - Vpc
    - Elastic IP: click **Allocate Elastic IP** and choose it
    - Create
2. After NAT is **Available**, edit **private-rt** routes:
    - Select `private-rt` → **Routes → Edit routes → Add route**
        - Destination: `0.0.0.0/0`
        - Target: `NAT Gateway (project-nat)`
    - Save

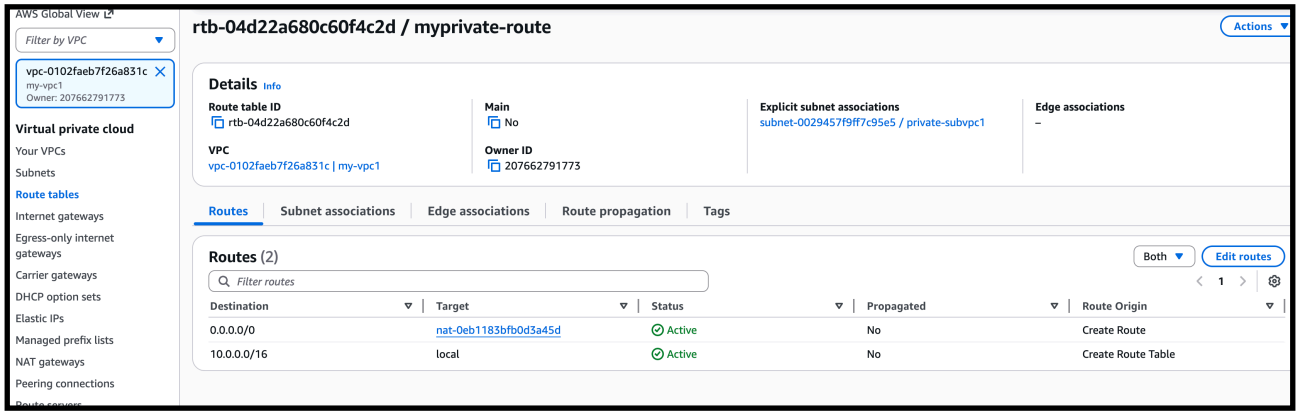
Now **private subnet** can go out to internet (yum/apt updates) but internet cannot directly reach it.



—> go to private route and attach the nat gateway



—-> nat gate way is attached to the private route table



6)Create two instances, one in the public subnet and one in the private subnet.

Steps:

—>Create EC2 instances (1 public, 1 private)

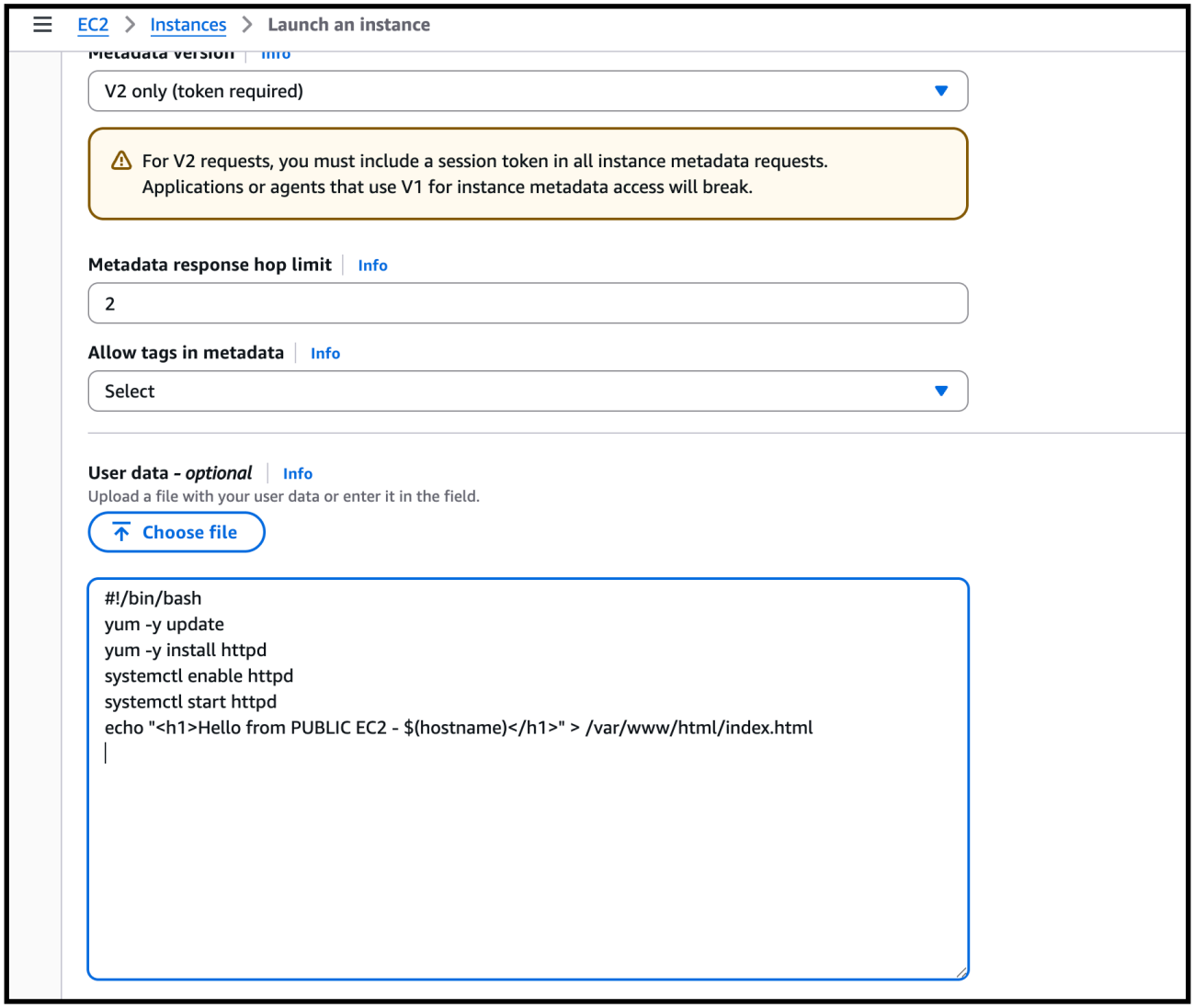
—> select the vpc which we created and for public ec2 select public subnet

—> and for same in private instance choose created vpc and select the private subnet of the vpc

—> and launch the instance

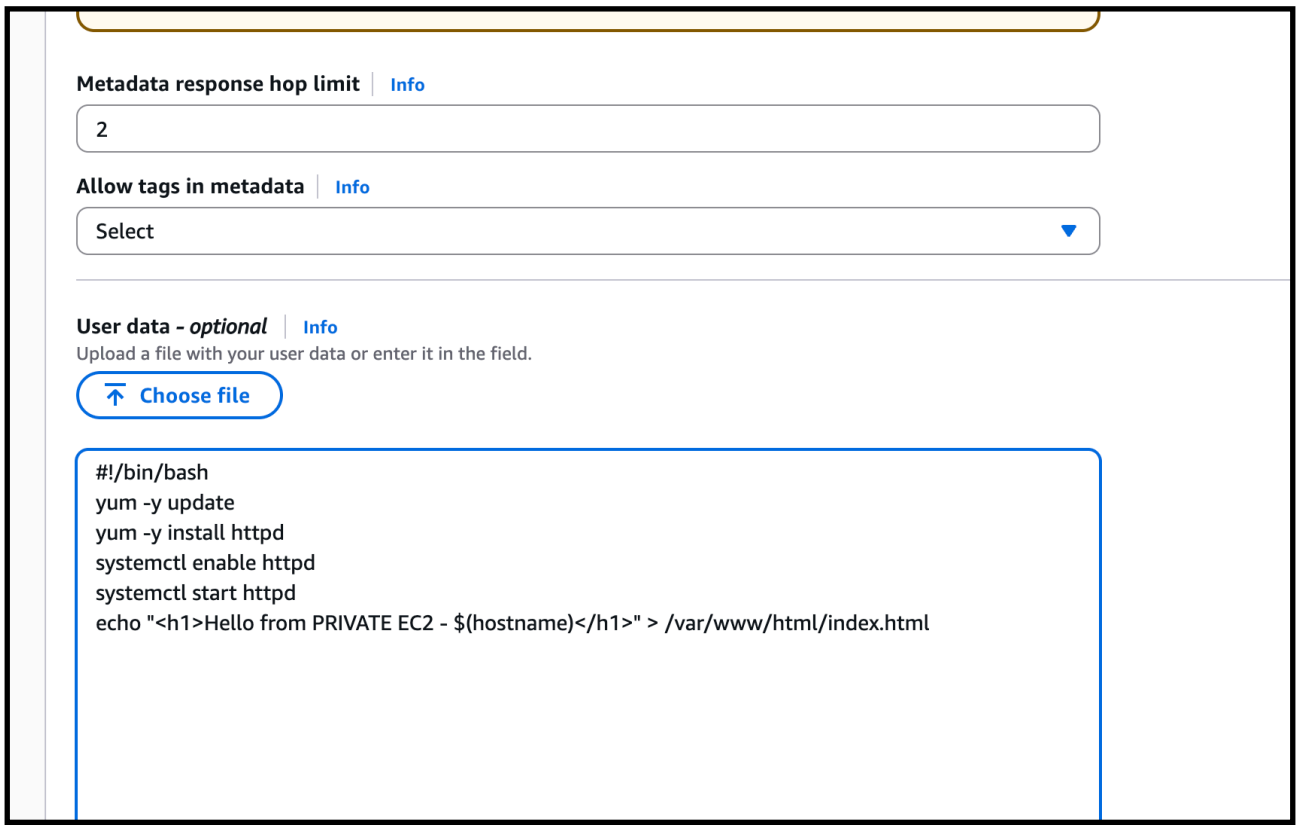
—> before launching I have added the script for installing httpd apache in data centre from task 7

7)Deploy Apache server on both EC2 instances with a sample index.html file.

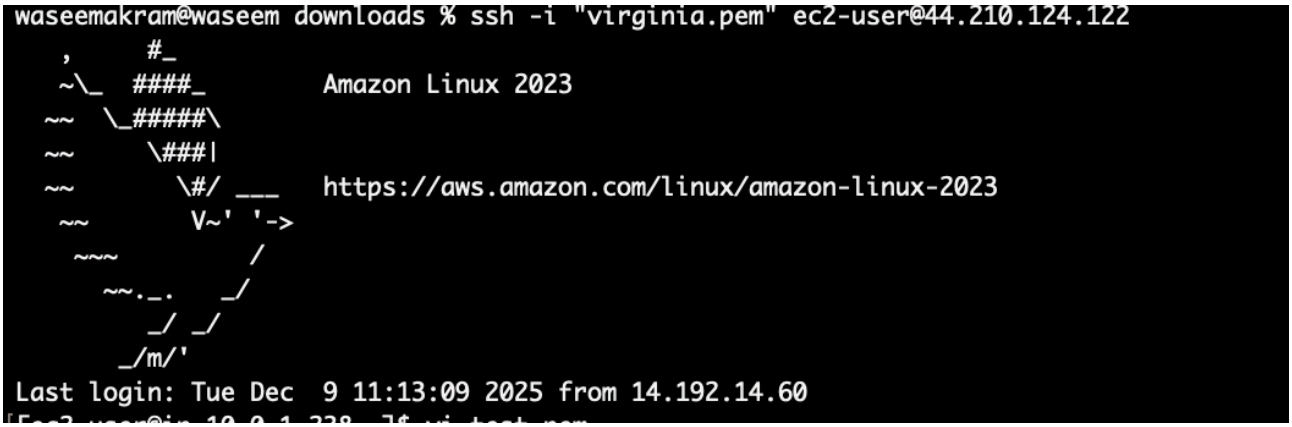


—> for public ec2 added the above script

—> for private ec2 added the below data



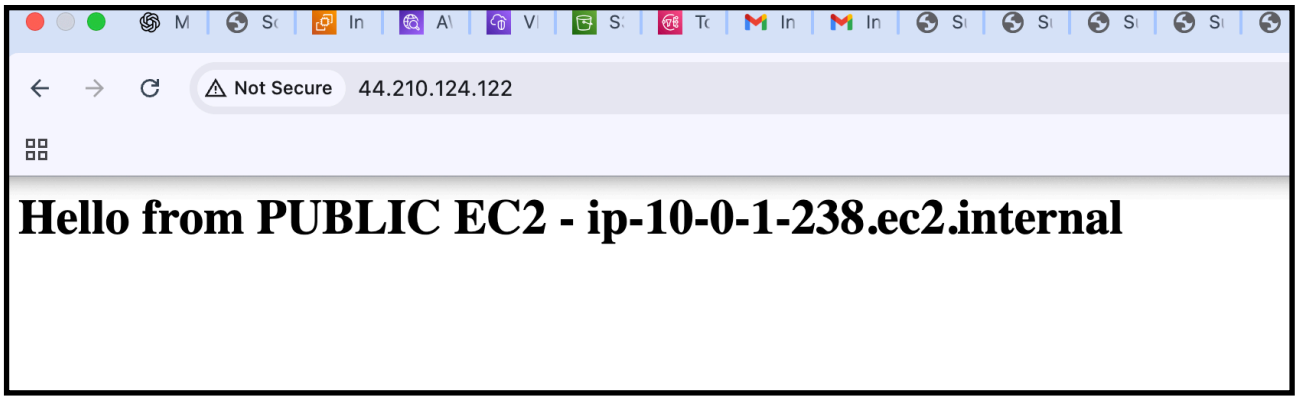
—> now to check the httpd index.html page is running login to the public ec2



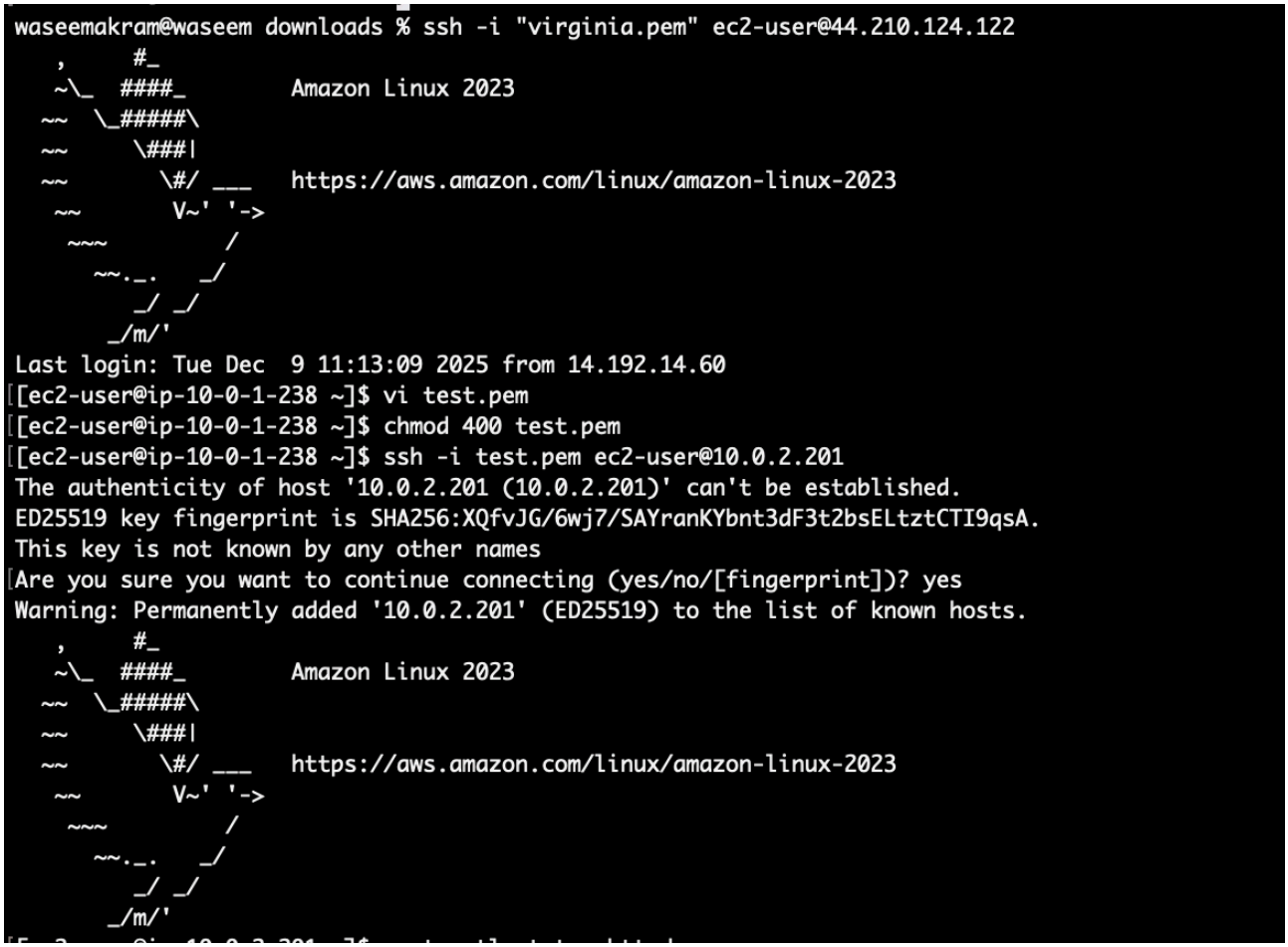
—-> we can see through image the apache is running

```
Warning: Permanently added '44.210.124.122' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-238 ~]$ systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Tue 2025-12-09 11:12:13 UTC; 1min 9s ago
       Docs: man:httpd.service(8)
   Main PID: 3194 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 1067)
     Memory: 13.3M
        CPU: 123ms
     CGroup: /system.slice/httpd.service
             ├─3194 /usr/sbin/httpd -DFOREGROUND
             ├─3288 /usr/sbin/httpd -DFOREGROUND
             ├─3294 /usr/sbin/httpd -DFOREGROUND
             ├─3295 /usr/sbin/httpd -DFOREGROUND
```

—-> now browse the public ip of the public ec2 to check index.html sample page

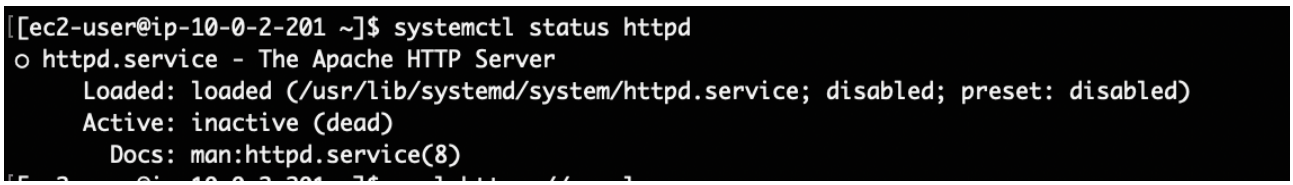


—> now lets login to our private ec2
—-> step to login in private ec2
—>first we have to copy the pem key by cat and
—> create a file name test.pem in public instance
—-> and add the pem key content inside the test.pem file
—> give chmod 400 for permission



—-> and ssh -I “test.pem” ec2-user@<private ip of private route>

—> by this we can successfully login to aur private ec2
—> now to check if it has httpd as we have connected nat gate way I should have internet access



—> httpd is available in side our private ec2

—> lets check the sample index.html of the private ec2 httpd

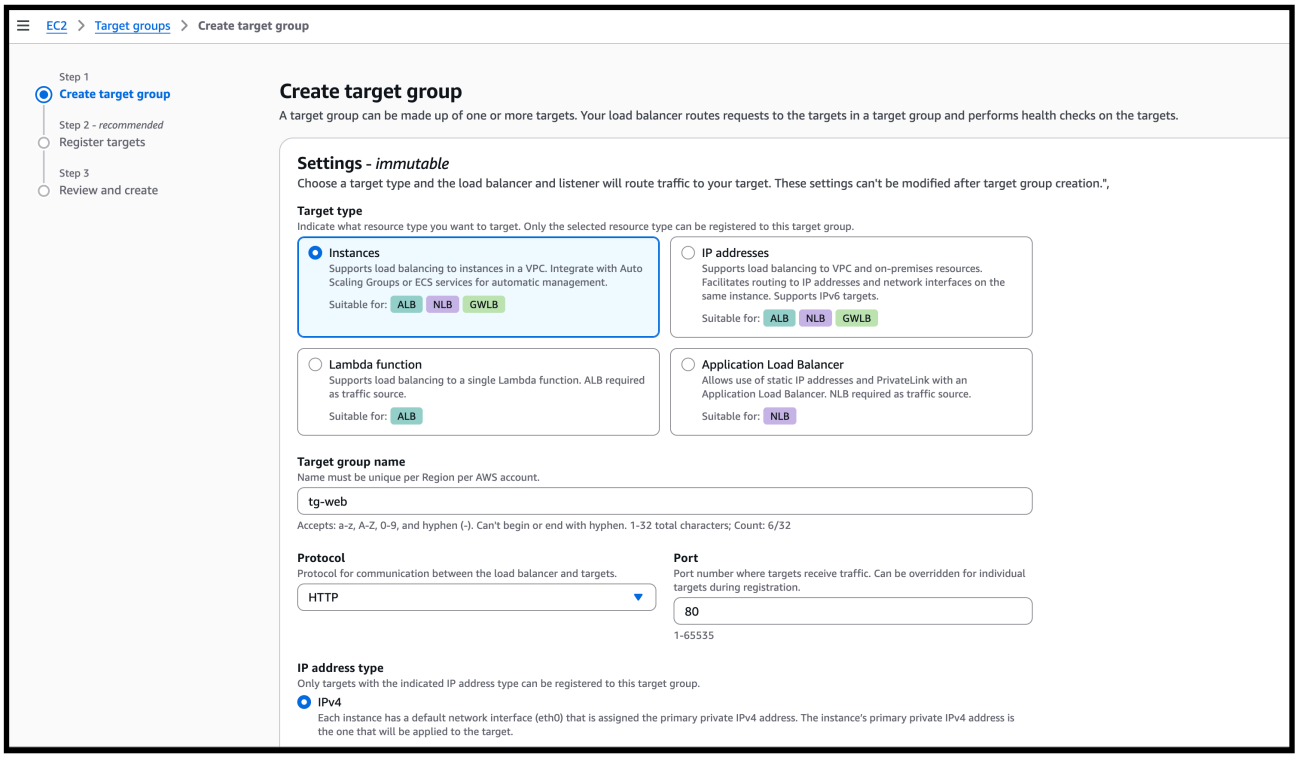
```
[[ec2-user@ip-10-0-2-201 ~]$ curl 10.0.2.201
<h1>Hello from PRIVATE EC2 - ip-10-0-2-201.ec2.internal</h1>
[ec2-user@ip-10-0-2-201 ~]$
```

8)Create one application load balancer and attach it to both EC2 instances.

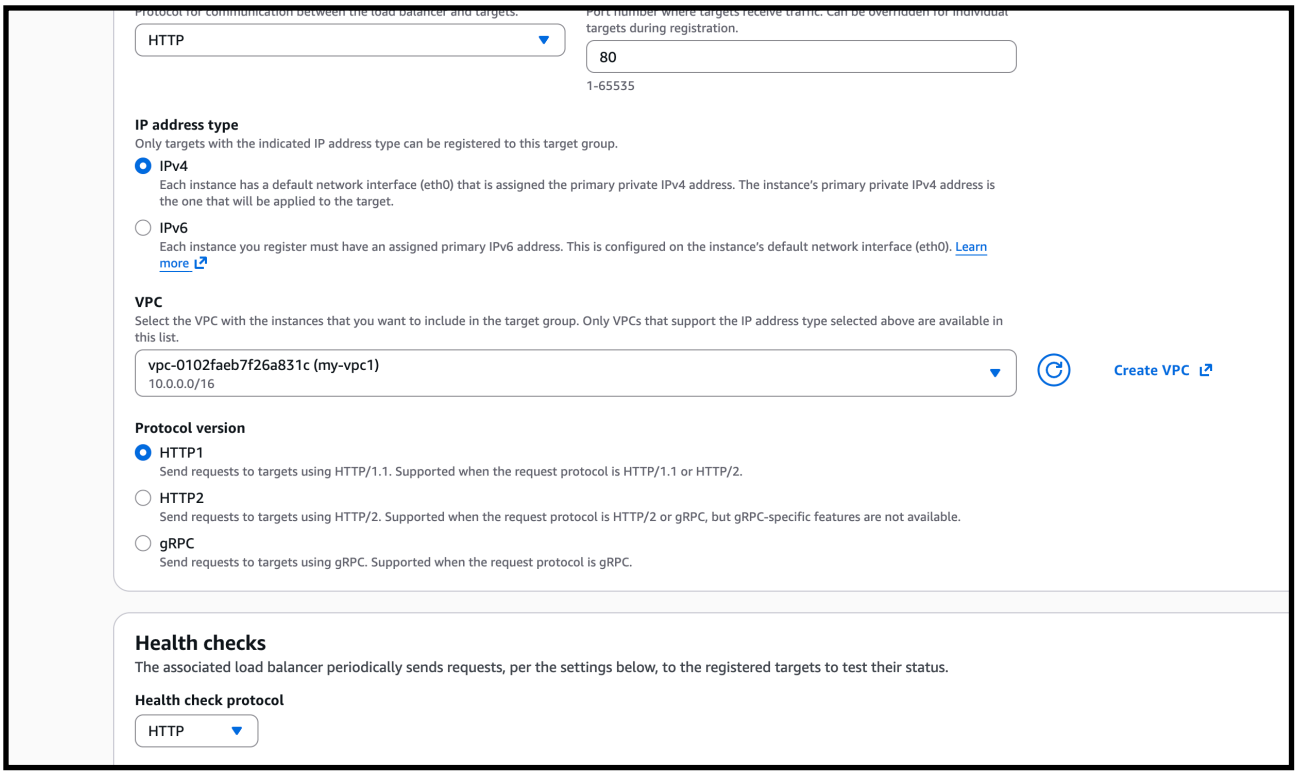
—> first create a target group for the load balancer

Step 1: Create Target Group

1. Go to **EC2 → Load Balancing → Target Groups**
2. Click **Create target group**
3. Choose **Instances**
4. Target group name: `my-target-group`
5. Protocol: **HTTP**
6. Port: **80**
7. VPC: select **your VPC**



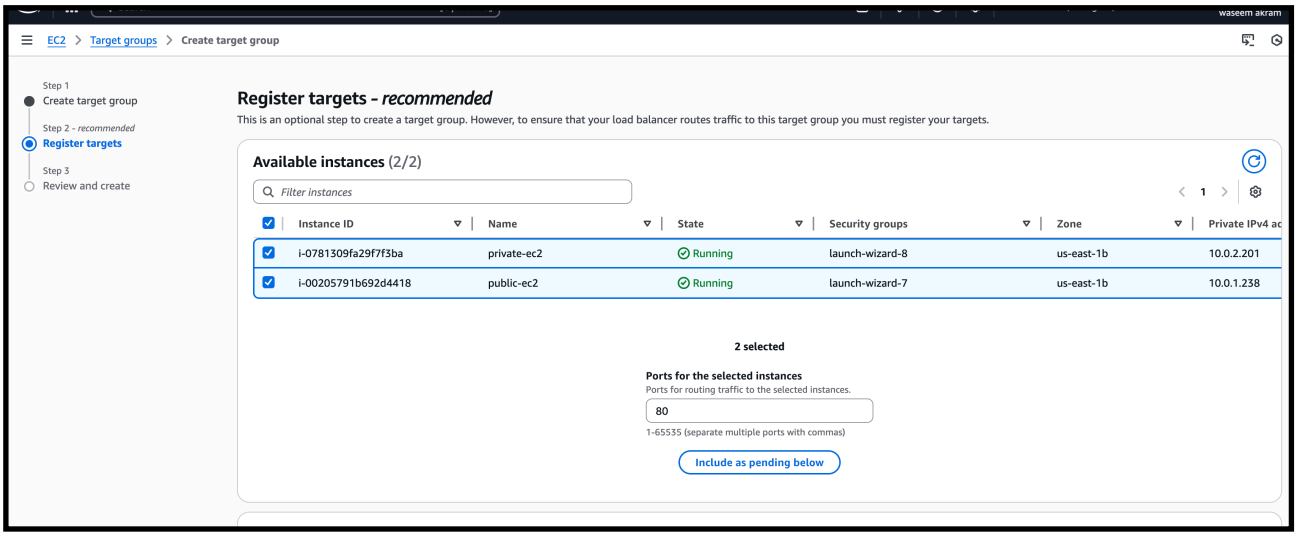
—> After that click next and you will be in register target step and we have to add the instances in register target

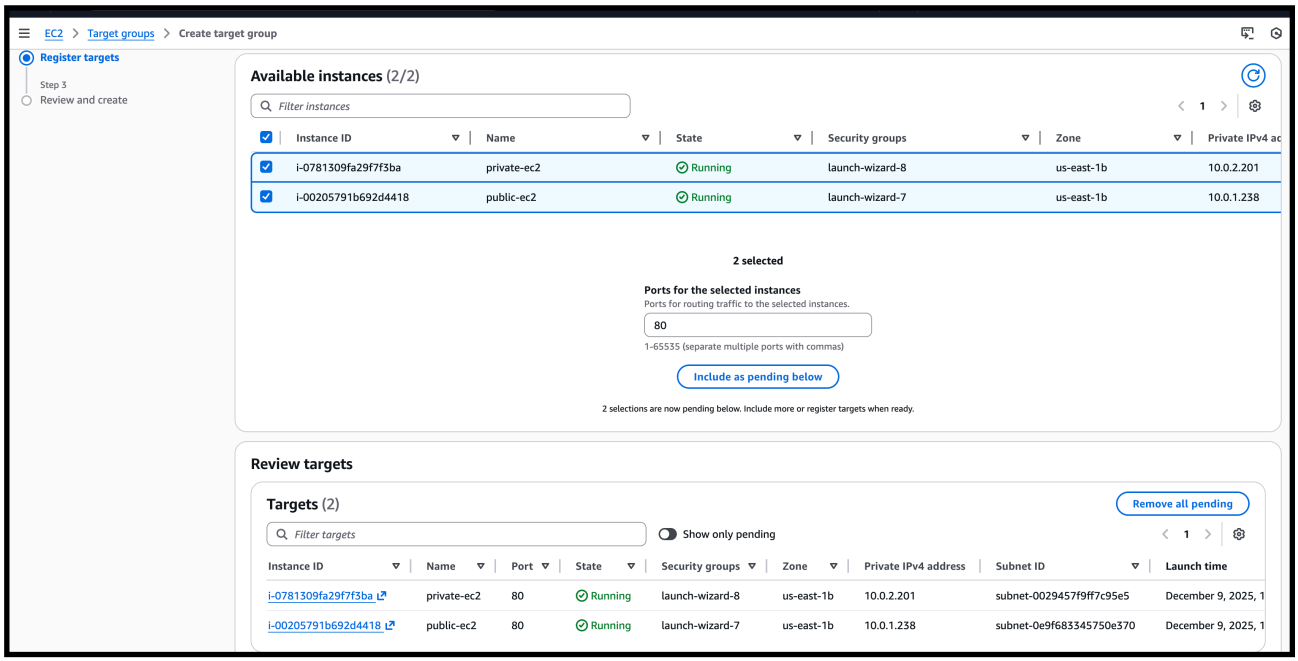


Register both instances:

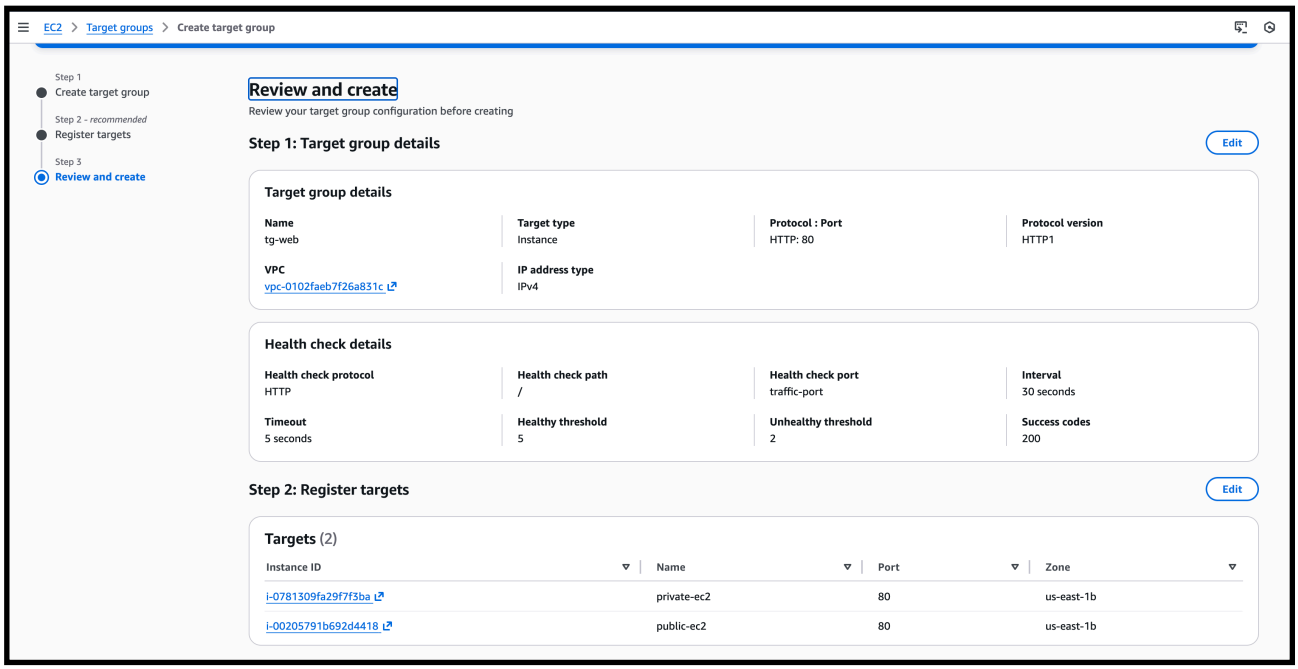
- public-ec2
- private-ec2

Click **Create target group**

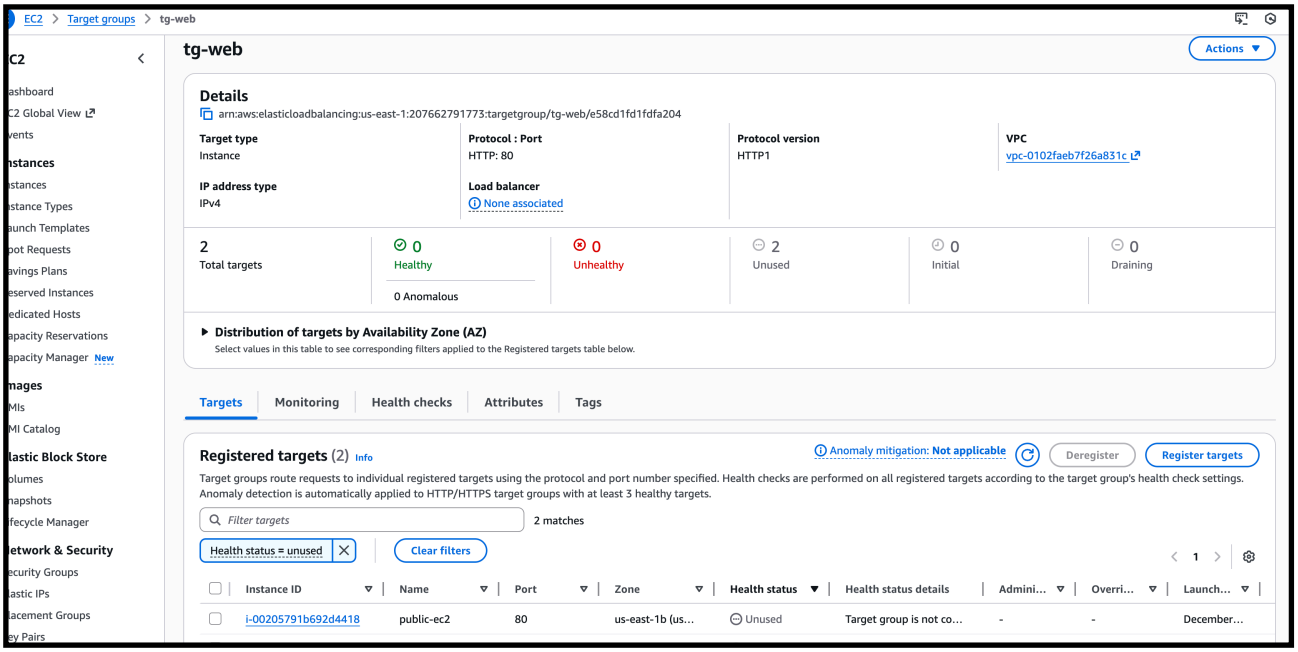




—-> after clicking next we will on review Stage of the target groups after review we have to simply click create target group



—> so currently we have created a “tg-web” named target groups



Now we have to create load balancer

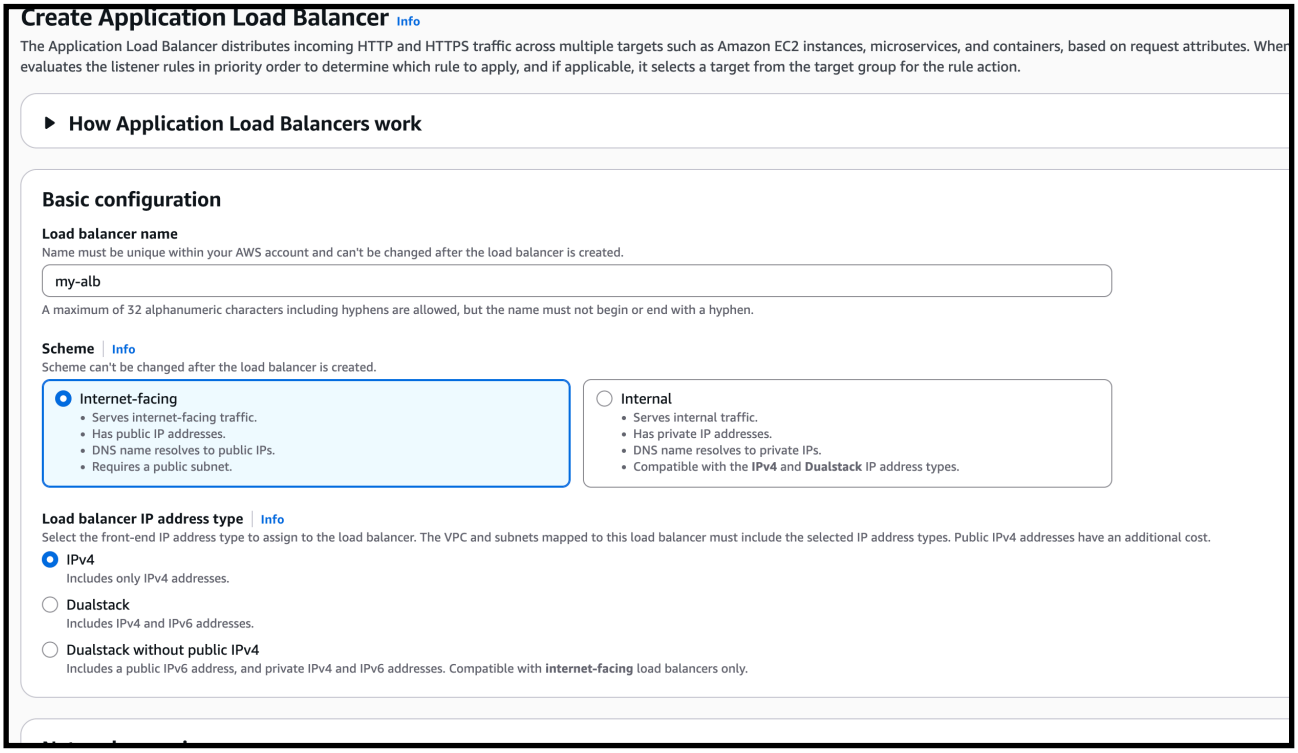
—->

Step 2: Create ALB

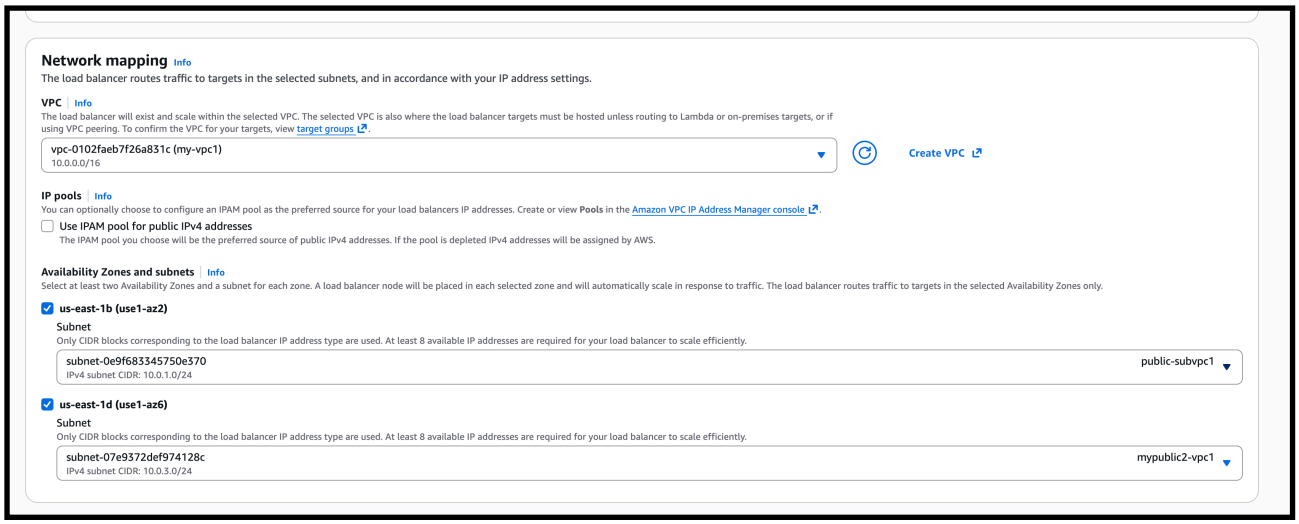
1. Go to **EC2 → Load Balancers**
2. Click **Create Load Balancer → Application Load Balancer**
3. Name:

   `my-alb`
4.
5. Scheme: **Internet-facing**
6. IP type: **IPv4**
7. Select **2 public subnets** (must have IGW route)
8. Security group:
   - Allow **HTTP (80)** from anywhere

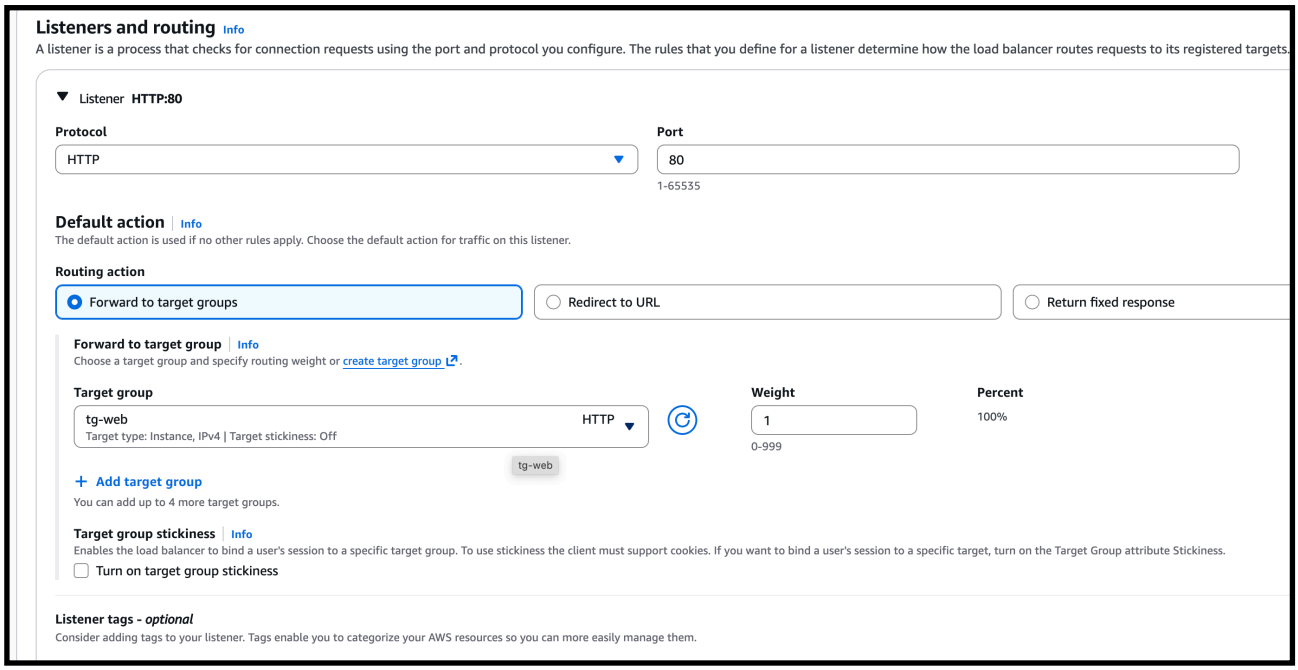
9. Listener:
    - HTTP:80 → select **my-target-group**
10. Click **Create Load Balancer**



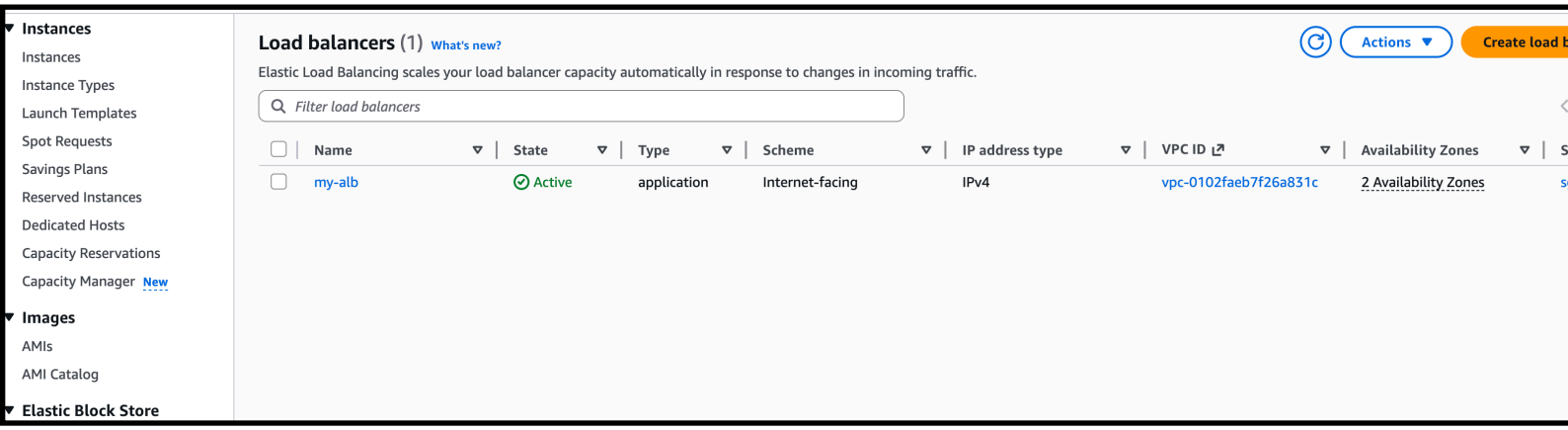
—> select the vpc which we created and add its availability zones Subnets



—> now attach the target group which we created “Tg-web”



—> and next step click create



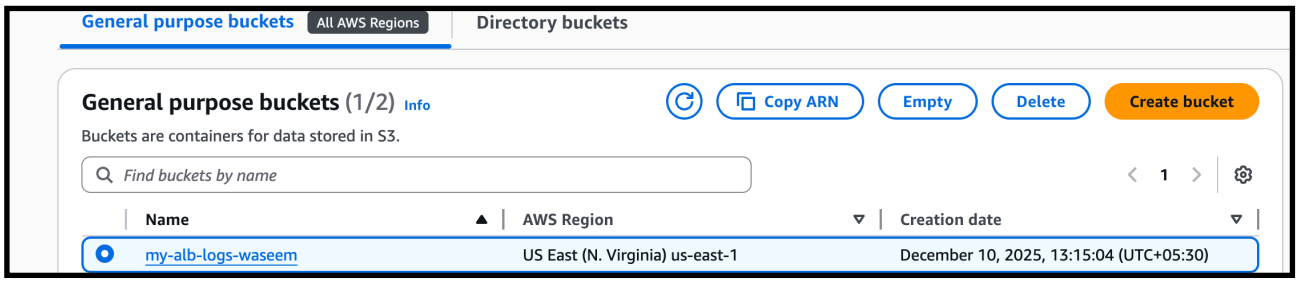
By this we have successfully created a application load balancer for and we have attached it with subnets (ec2)

9) Store application load balancer logs in S3.

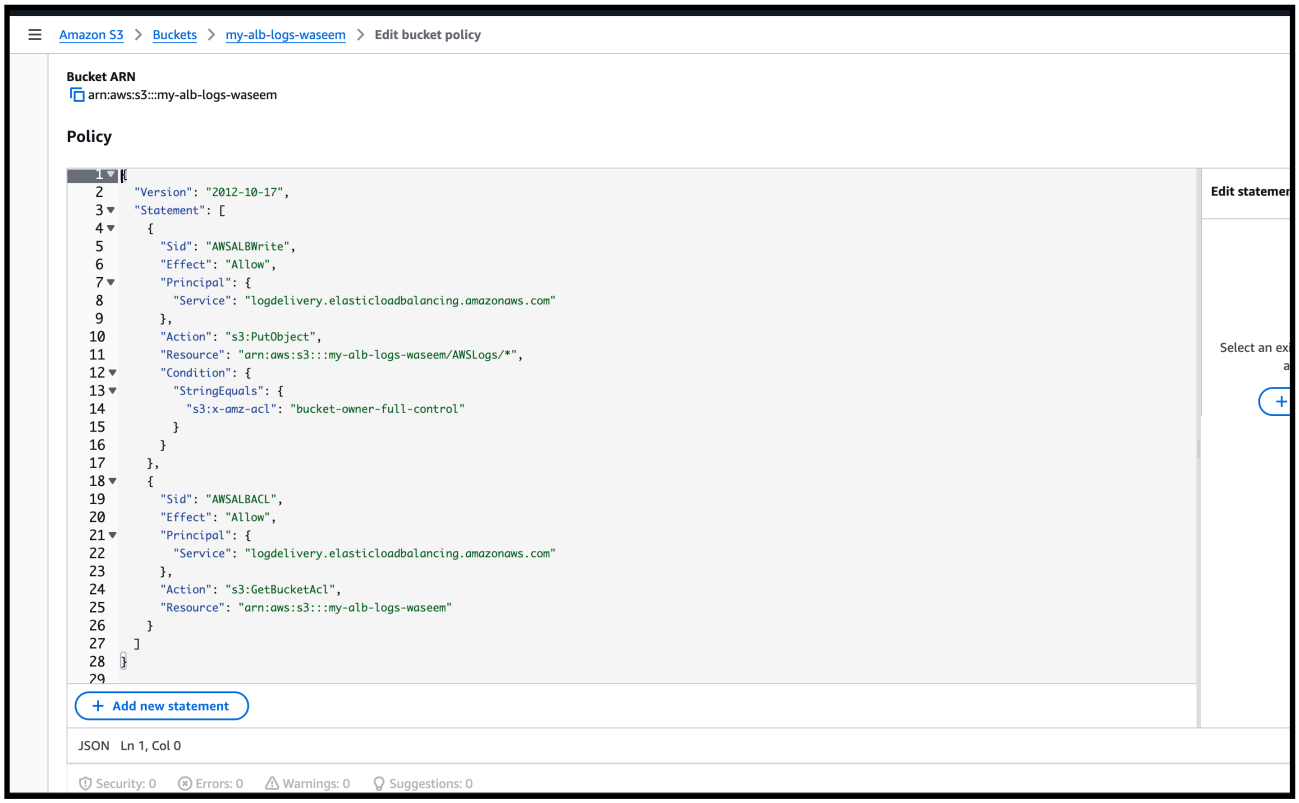
—> to store the logs of load balancer we have to create a s3 bucket Named with that load balancer

—> I given the “my-alb-logs-waseem” as bucket name

1. Block public access → Keep ON (default)
2. Create bucket



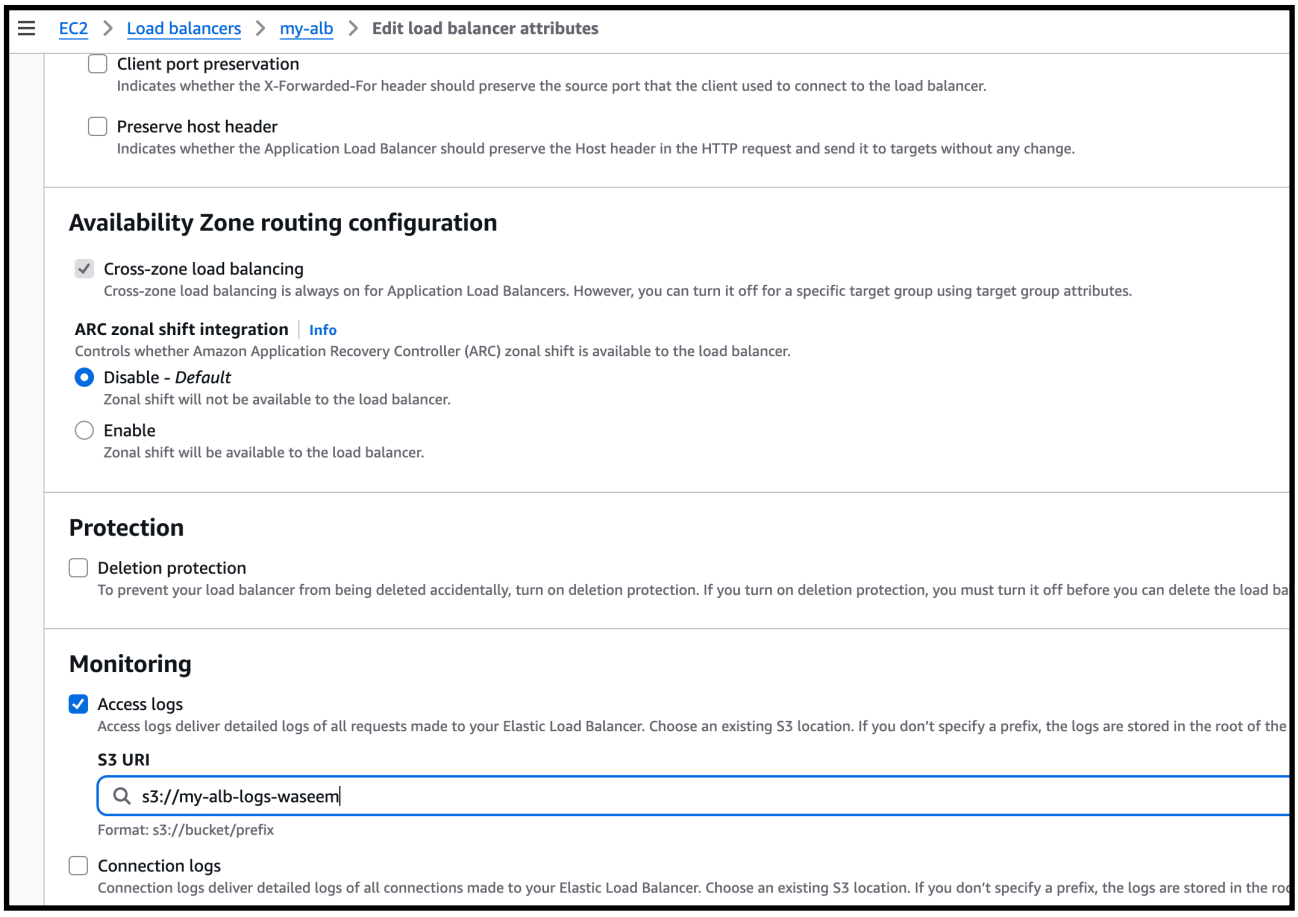
—> give the bucket policy for storing logs from load balancer



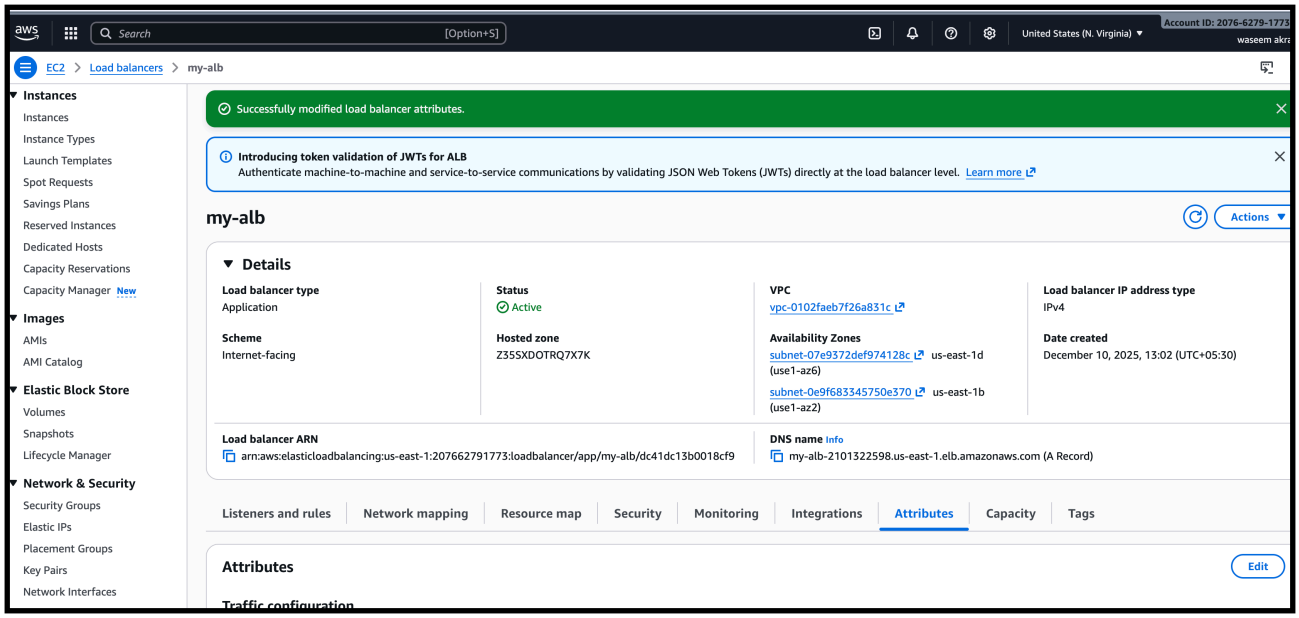
—-> now go to the load balancer —> my-alb—> and look for attributes

Enable ALB Logs

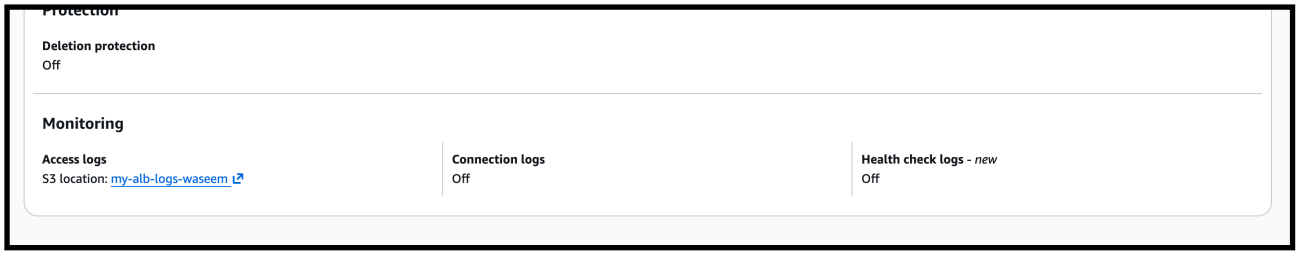
1. Go to **EC2 → Load balancers → my-alb**
2. Click **Attributes**
3. Enable:
    - **Access Logs → ON**
4. Provide S3 bucket name: browse the name of the bucket which we created



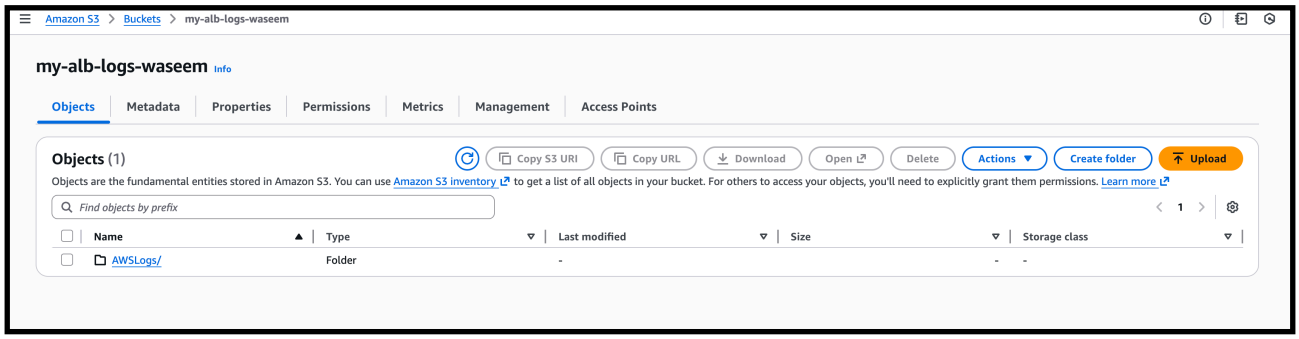
—-> after this step click next

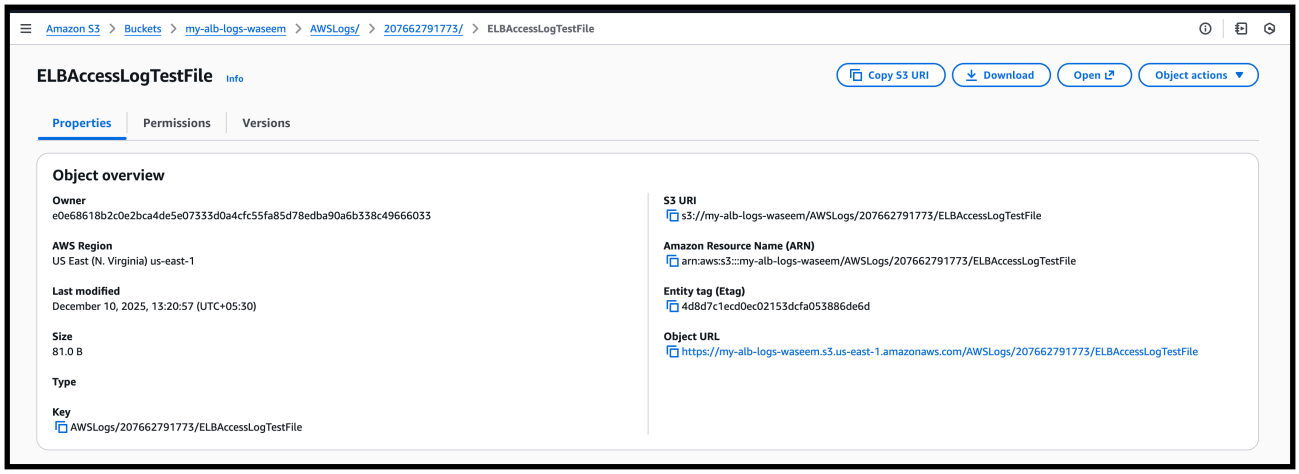


Done.
—> After few minutes logs will appear inside S3.



—> to view this logs go to the bucket and view object





—-> open all logs till end
—-> by this our load balancer logs are being stored in our s3 bucket

10) Store the VPC flow logs in a CloudWatch log group.

It will record all incoming + outgoing traffic inside your VPC
(like who connected, from which IP, which port, etc.)

—steps:~

✔ Create VPC Flow Logs
✔ Send flow logs into **CloudWatch Log Group**
✔ Not to S3 bucket

1: Open VPC Console

Go to:

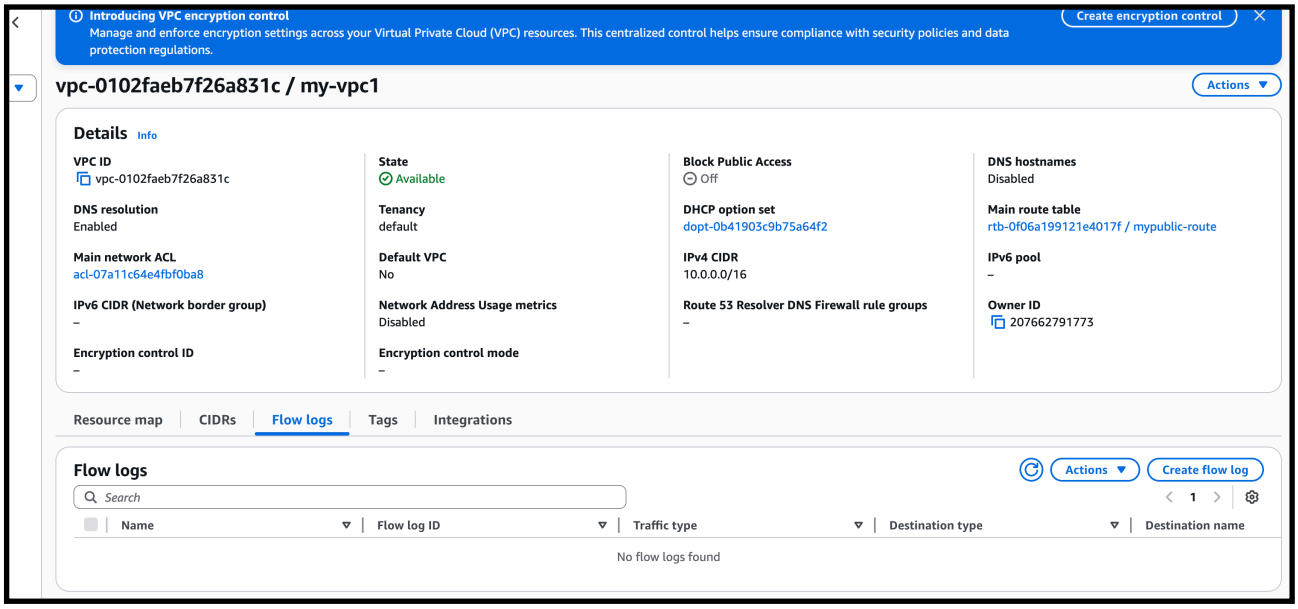
AWS → VPC → Your VPCs

Select **your VPC**

2: Create Flow Log

After selecting your VPC:

Left side → **Flow logs** → Click **Create flow log**



—>Correct Settings You Should Select

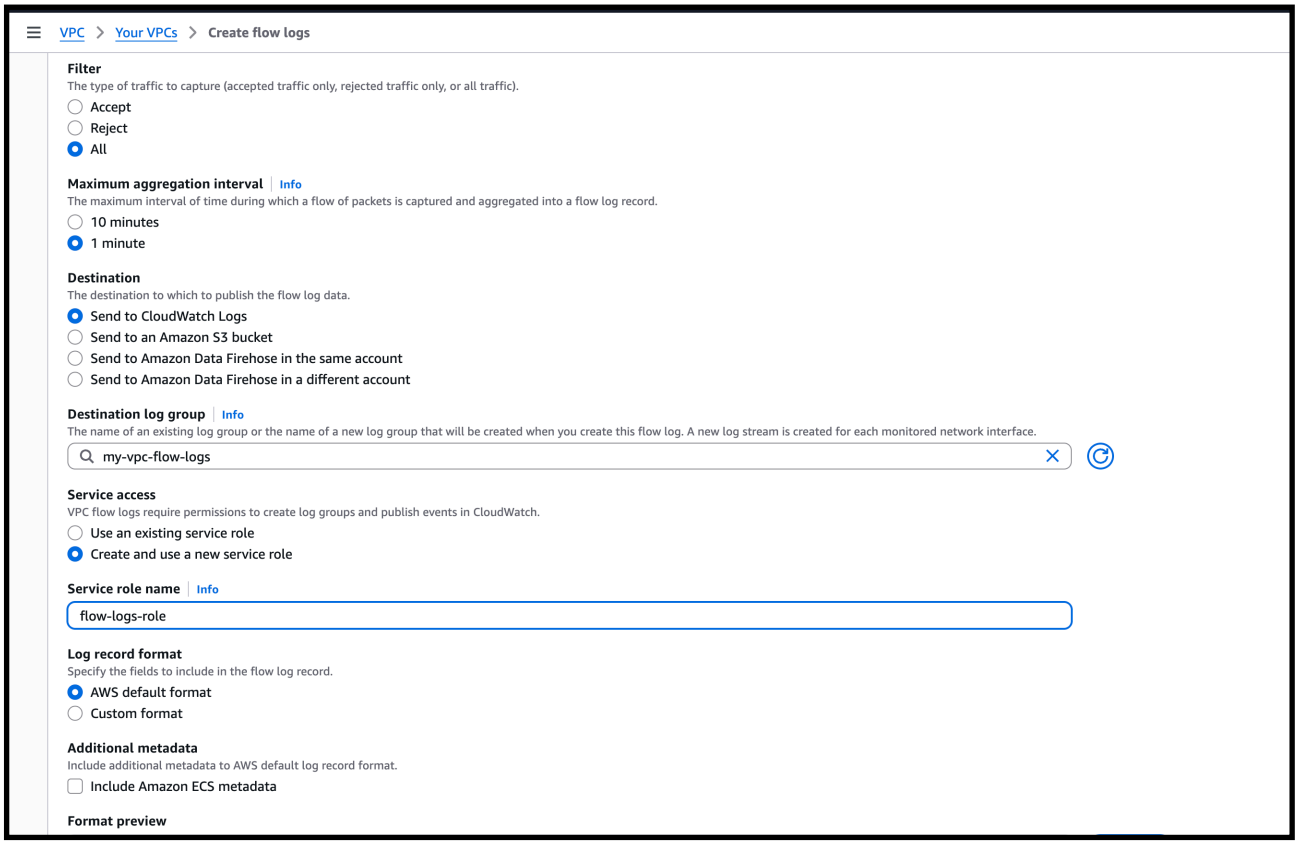
✔ **Filter: All**

✔**Max aggregation:1 minute**

✔**Destination :Send to CloudWatch Logs**

✔ **Destination log group : my-vpc-flow-logs**

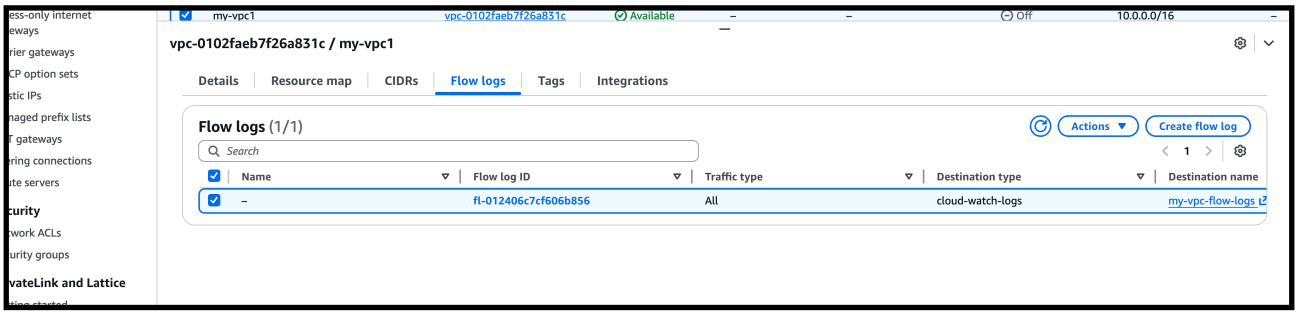
==> give a role name and keep other settings default



Step 5: Click Create

That’s it.

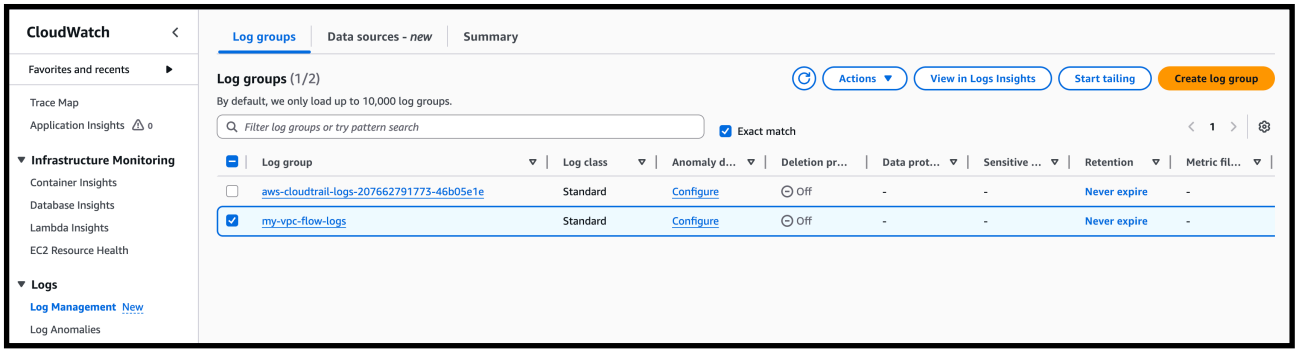
Your VPC Flow Logs will start coming inside CloudWatch log group.

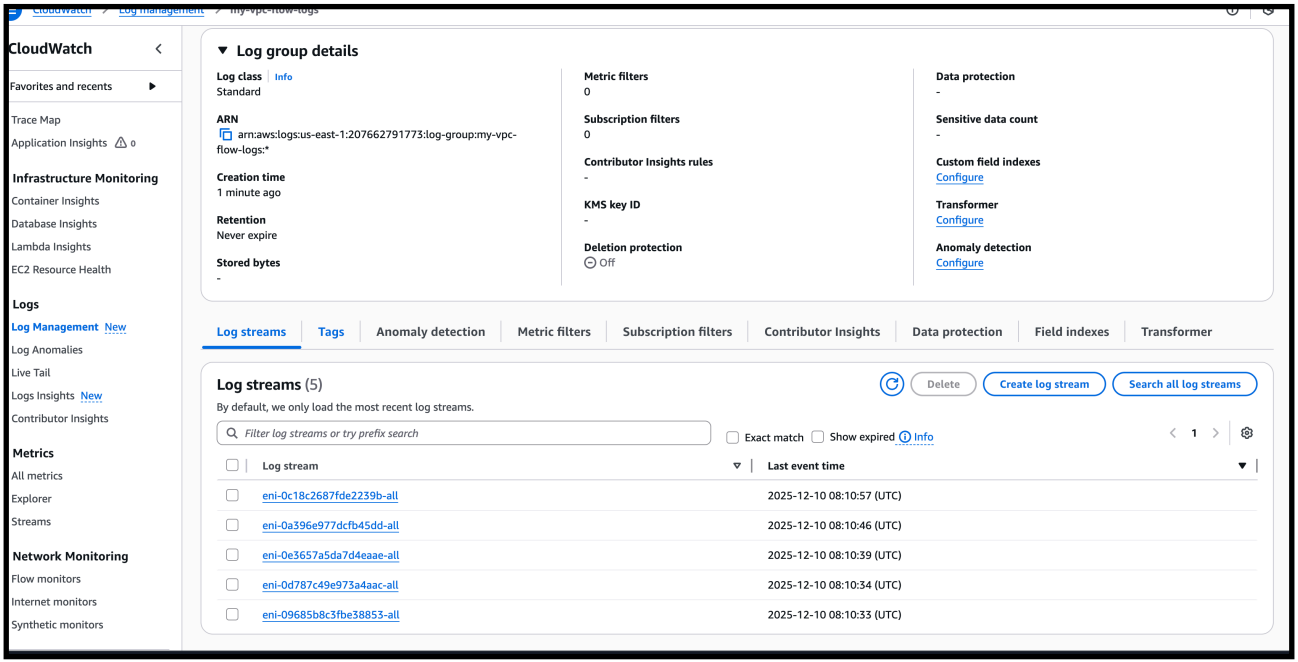


—> What You Should See Next

Inside CloudWatch:

Log Groups → `/my-vpc-flow-logs`





—> VPC Flow Logs are WORKING properly and being sent to **CloudWatch Log Group `/my-vpc-flow-logs`**.

11)Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.

must create a dashboard with:

(A) CPU Utilization Monitoring for both EC2 Instances

(B) Apache HTTPD Service Monitoring

Let's start step-by-step.

—>Go to CloudWatch → Dashboards

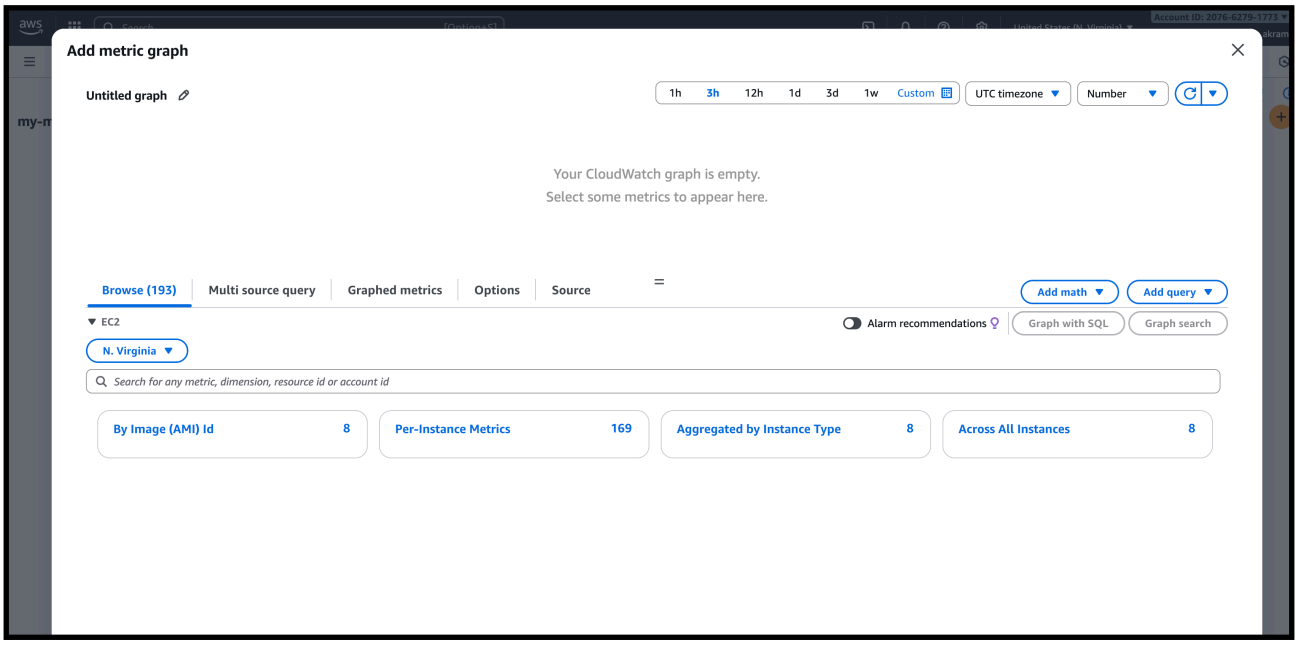
Left side → **Dashboards** → **Create dashboard**

Name it:

`my-monitor-dashboard`

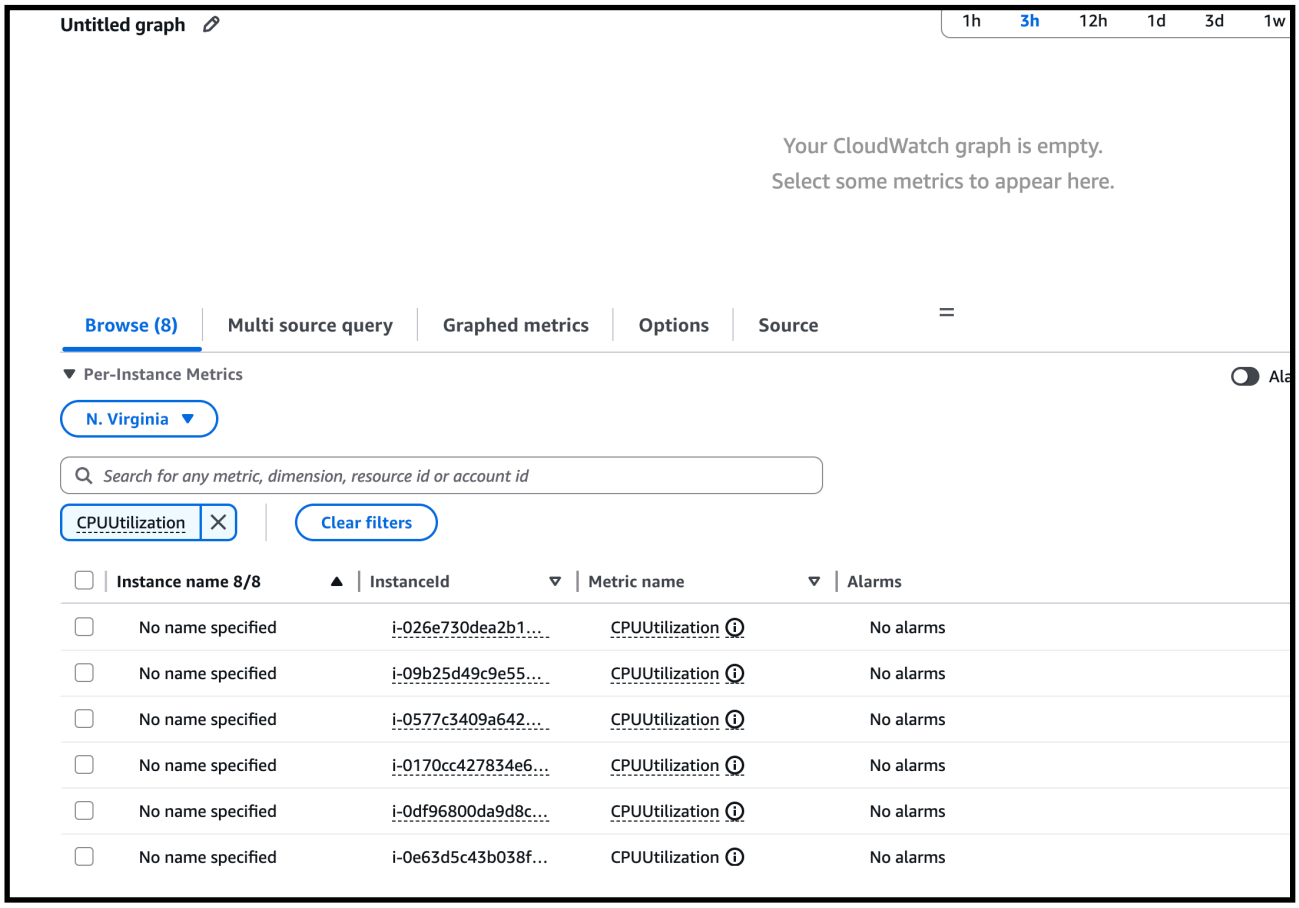
—> Add Widget → Line Graph → CPU Metrics

→ Click **Add widget**
→ Choose **number**
→ Choose **Metrics**
→ EC2 → Per-Instance Metrics

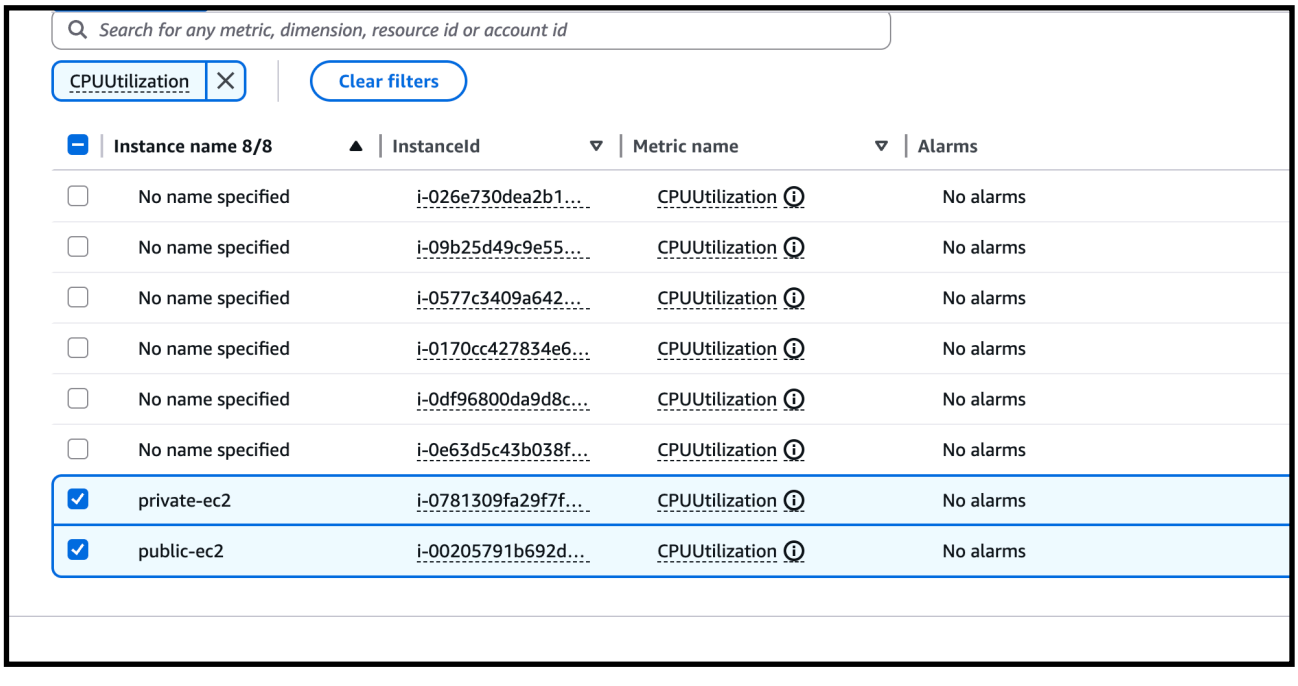


→ Select BOTH instances:

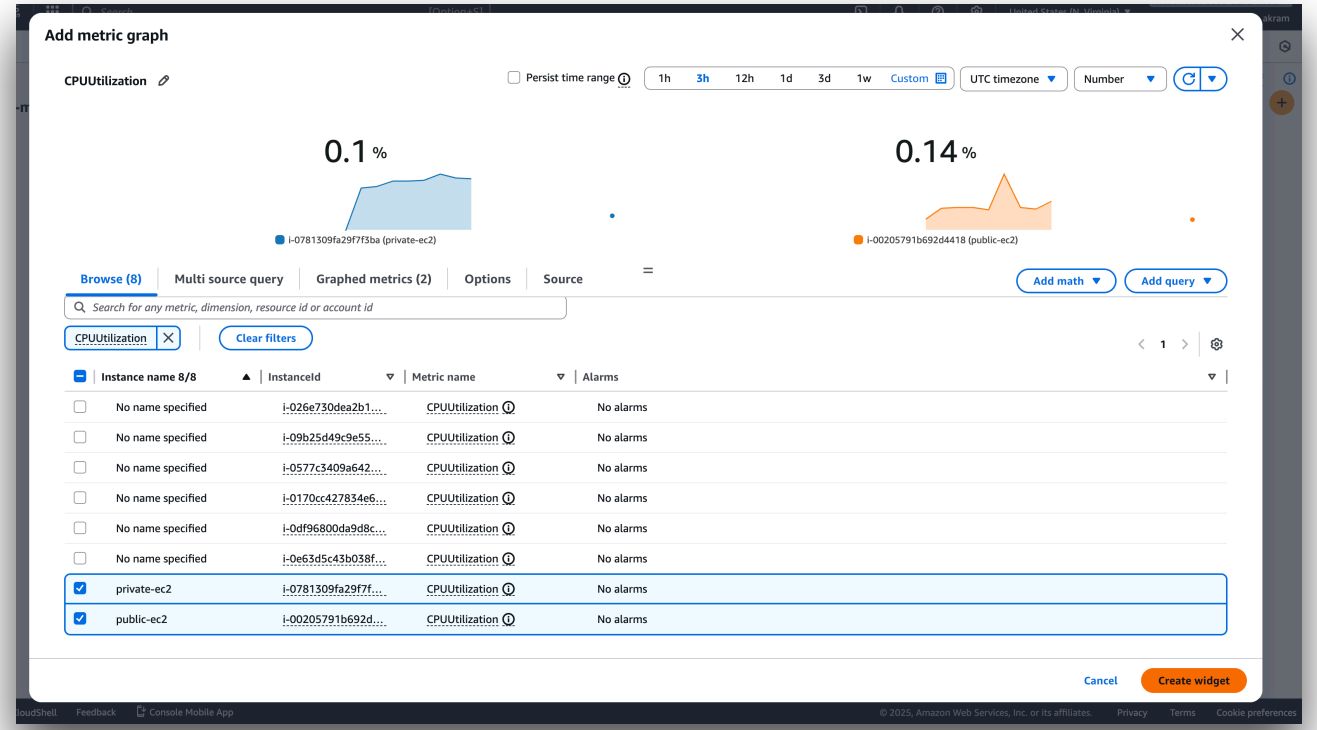
- private-ec2 → CPUUtilization
- public-ec2 → CPUUtilization

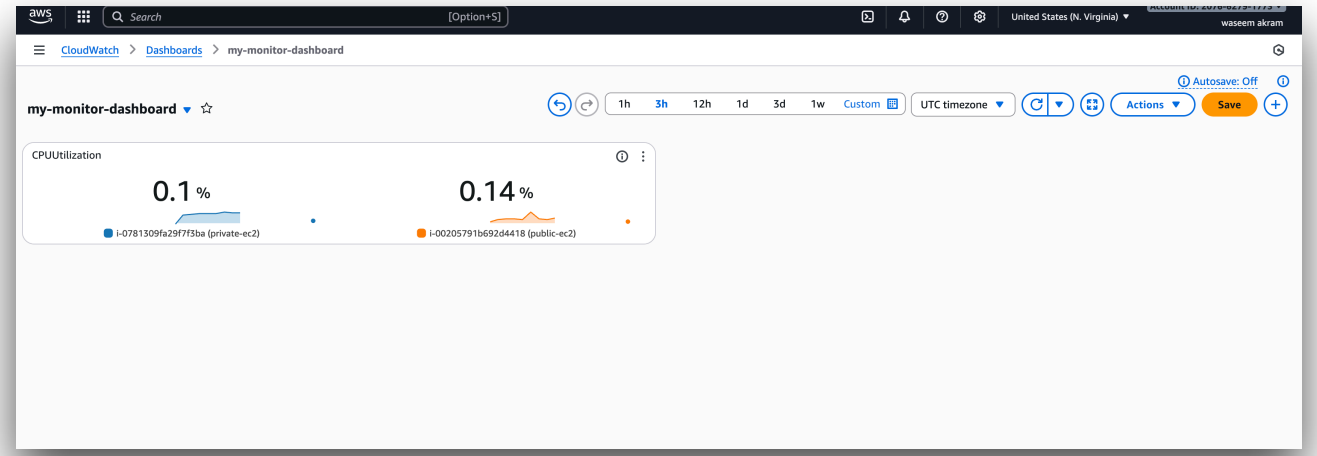


—> select the both instances



—> click next





(A) CPU Utilization Monitoring for both EC2 Instances

We have created the dashboard for monitoring the cpuutilization of the two instances

B)—> Now lets create a dash board for the httpd monitoring

—-> go inside the public ec2 instance and create a script to create a matric of the httpd

—> this below script will create a matric for httpd

—>. “opt/apache-monitor.sh” inside this add the script

```
#!/bin/bash

if systemctl is-active --quiet httpd; then
  STATUS=1
else
  STATUS=0
fi

aws cloudwatch put-metric-data \
  --namespace "CustomApache" \
  --metric-name "ApacheStatus" \
  --value $STATUS \
  --region us-east-1
~
~
~
~
~
~
~
~
~
~
```

—-> create a crontab for the this script file

Create cronjob to run every 1 minute

—> Then make it executable: sudo chmod +x /opt/apache-monitor.sh

```
Warning: Permanently added '3.93.79.221' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Wed Dec 10 07:57:26 2025 from 14.192.14.60
[ec2-user@ip-10-0-1-238 ~]$ ls
test.pem
[ec2-user@ip-10-0-1-238 ~]$ sudo vi /opt/apache-monitor.sh
[ec2-user@ip-10-0-1-238 ~]$ sudo chmod +x /opt/apache-monitor.sh
[ec2-user@ip-10-0-1-238 ~]$ sudo crontab -e
sudo: crontab: command not found
[ec2-user@ip-10-0-1-238 ~]$ sudo yum install cronie -y
Last metadata expiration check: 22:02:15 ago on Tue Dec  9 11:12:08 2025.
Dependencies resolved.
================================================================================
 Package                                                    Architecture
================================================================================
Installing:
 cronie                                                     x86_64
Installing dependencies:
 cronie-anacron                                             x86_64

Transaction Summary
================================================================================
Install  2 Packages
```

—-> below is the list of our cronjob

```
[ec2-user@ip-10-0-1-238 ~]$ sudo systemctl start crond
sudo systemctl enable crond
[[ec2-user@ip-10-0-1-238 ~]$ sudo crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[[ec2-user@ip-10-0-1-238 ~]$ sudo crontab -l
* * * * * /opt/apache-monitor.sh

[ec2-user@ip-10-0-1-238 ~]$
```

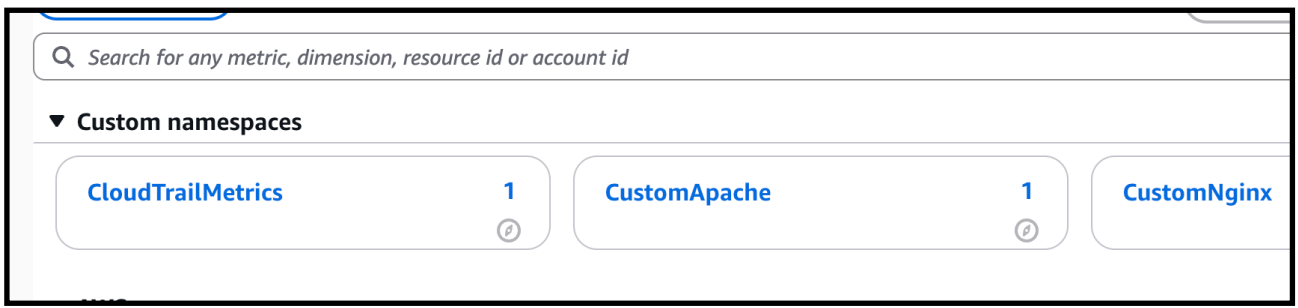
—-> after that we have to login to our aws account
—-> with aws configure
—-> and add all the access key details with region

```
[AWS Secret Access Key [None]: eo/IVvxKm8ZSzh0GlByif/L55gK1ilUbZIfUI+q7
[eo/IVvxKm8ZSzh0GlByif/L55gK1ilUbZIfUI+q7
[[ec2-user@ip-10-0-1-238 ~]$ aws configure
[AWS Access Key ID [None]: AKIATAWNKXROZALLH66G
[AWS Secret Access Key [None]: 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
[Default region name [None]: us-east-1
[Default output format [None]: json
[[ec2-user@ip-10-0-1-238 ~]$ sudo vi /opt/apache-monitor.sh
 [ec2-user@ip-10-0-1-238 ~]$ /opt/apache-monitor.sh
 [ec2-user@ip-10-0-1-238 ~]$
```

—>And later run the script
/opt/apache-monitor.sh

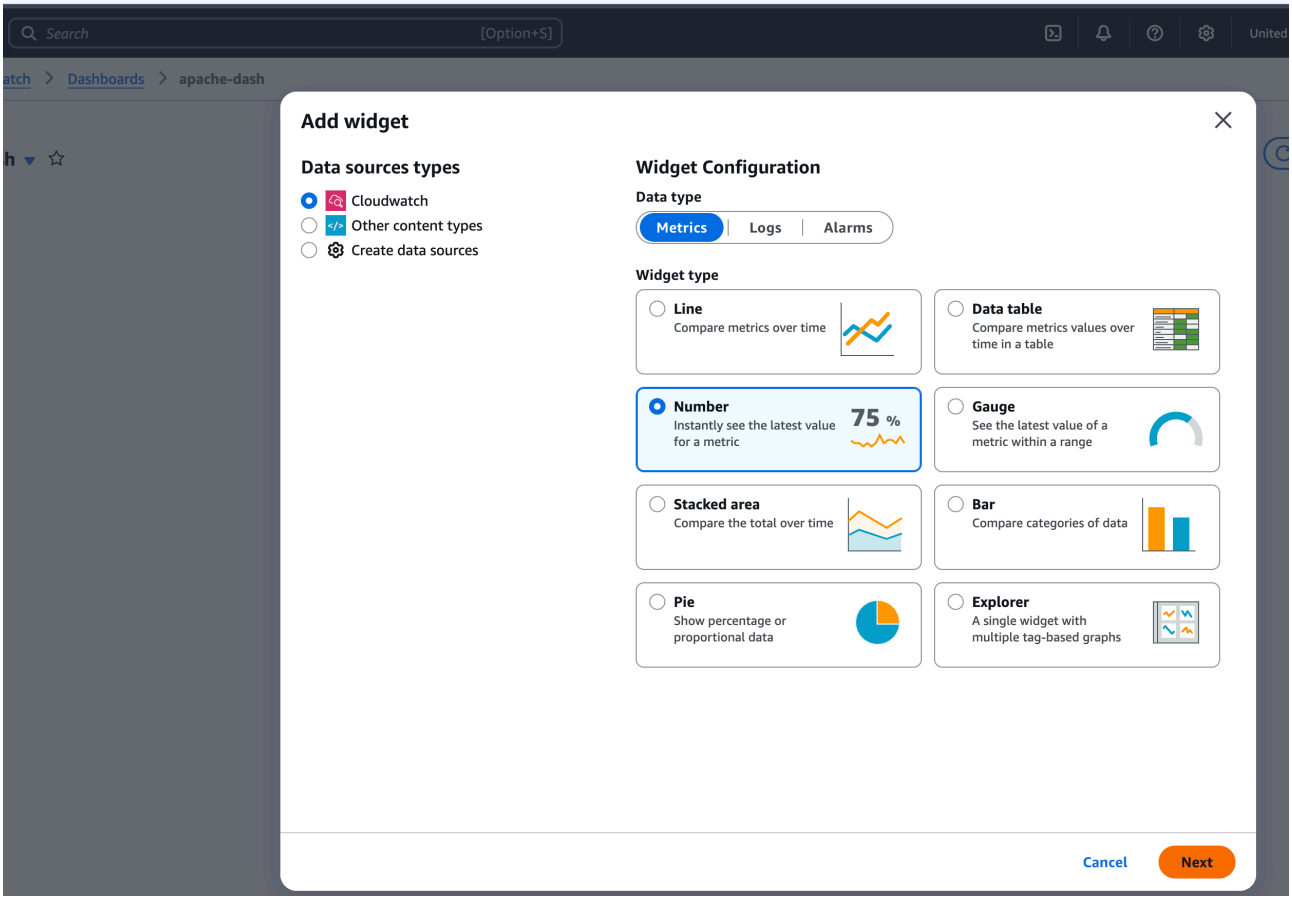
—-> after running the script our matric for httpd (apache) will be created

—-> to check go to cloudwatch—>all matric

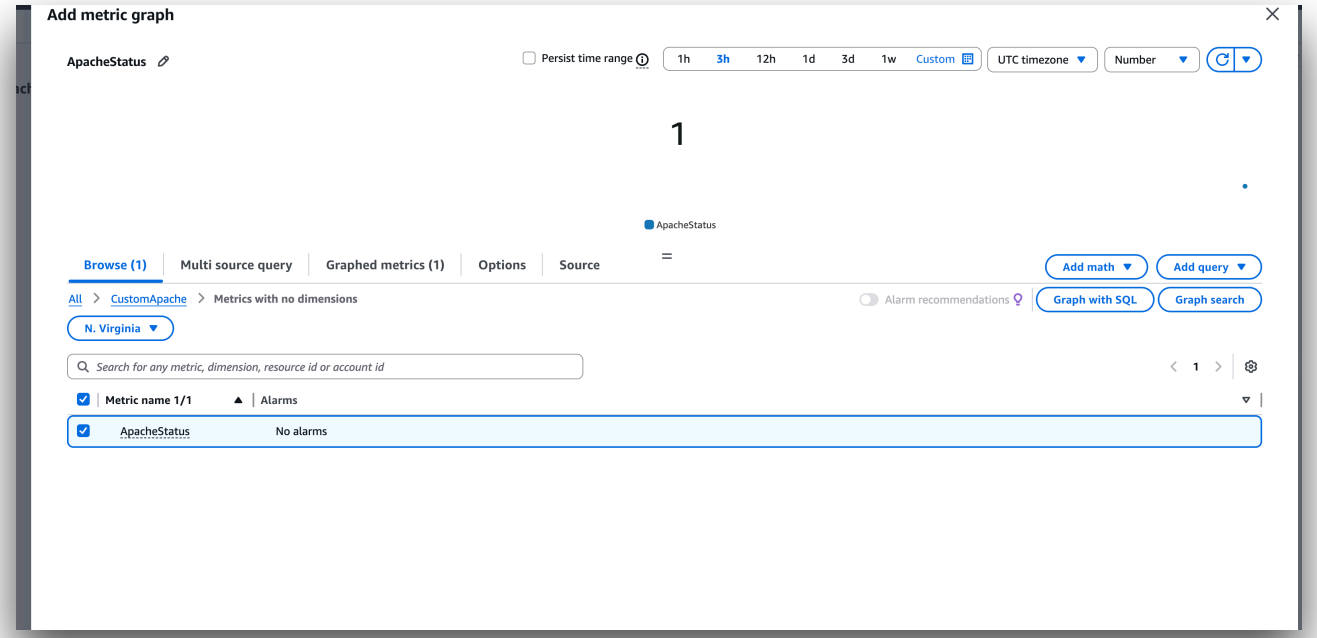


—-> now lets create a dash bord of the apache

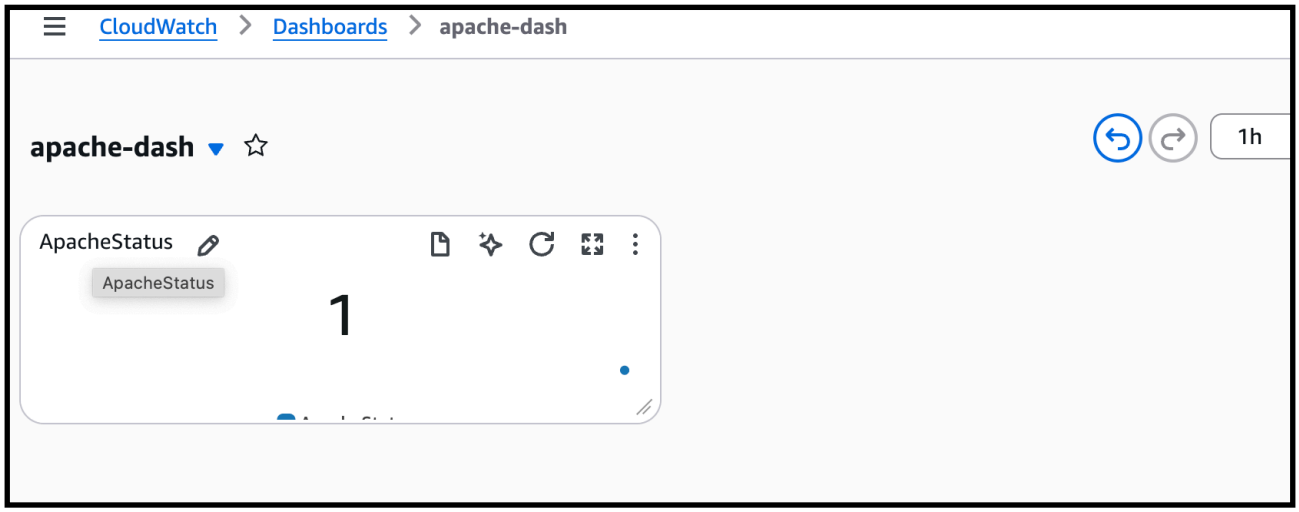
—-> dashbord —>create dashboard—>select the apache matric



—> And create



—-> we have now we can see the apache status as 1 because it is running



—> by this we have created dashboard for both

(A) CPU Utilization Monitoring for both EC2 Instances

(B) Apache HTTPD Service Monitoring

12) If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

To trigger auto scaling at 70% CPU:

We need 4 things:

1️⃣ **Launch Template** (your EC2 configuration)
2️⃣ **Auto Scaling Group (ASG)**
3️⃣ **Target Group already exists** (we used it for ALB) → we will attach ASG to this
4️⃣ **Scaling Policy: CPU > 70% → Add 1 instance**

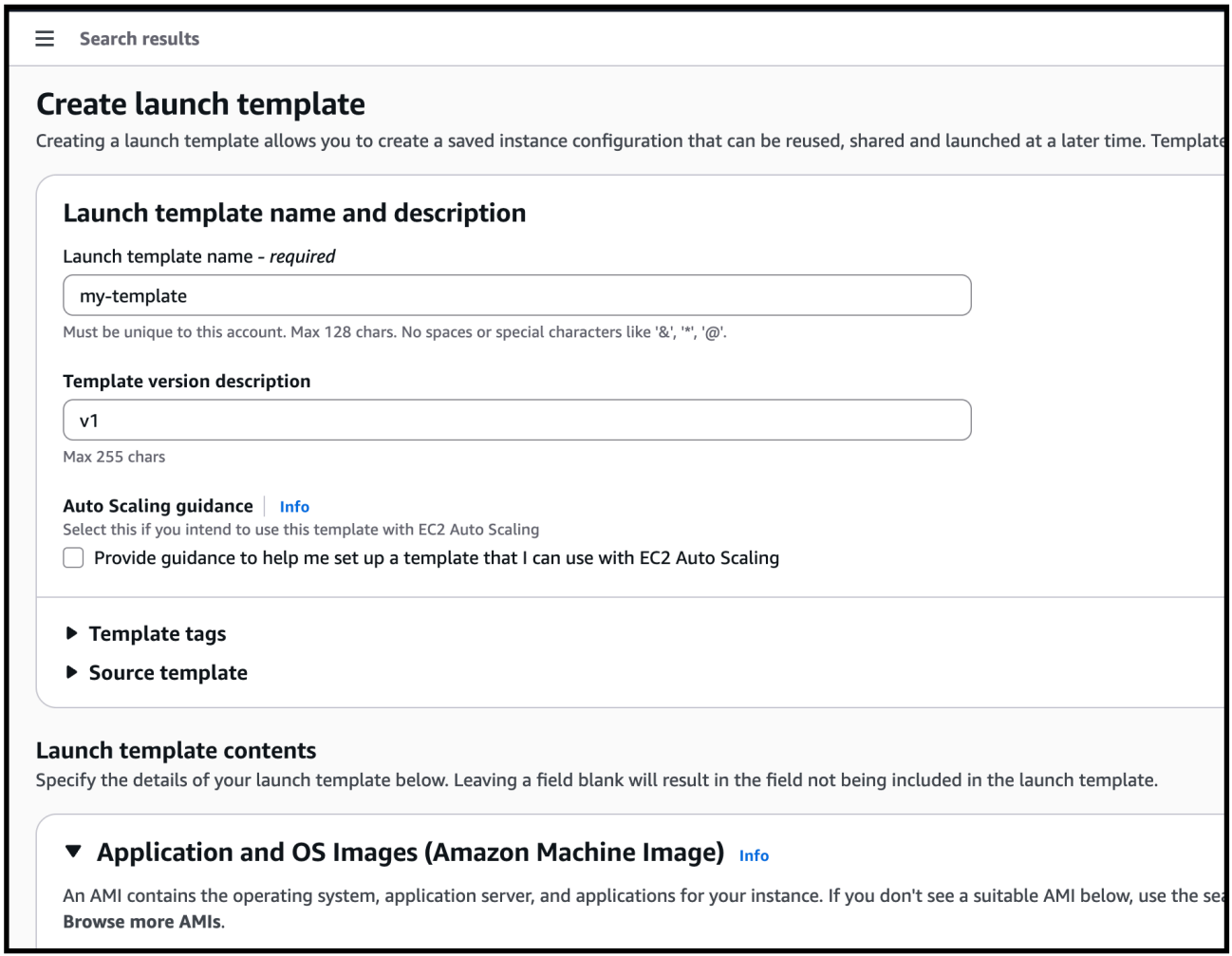
—> we have to create a launch tamplate

Create Launch Template

Go to:

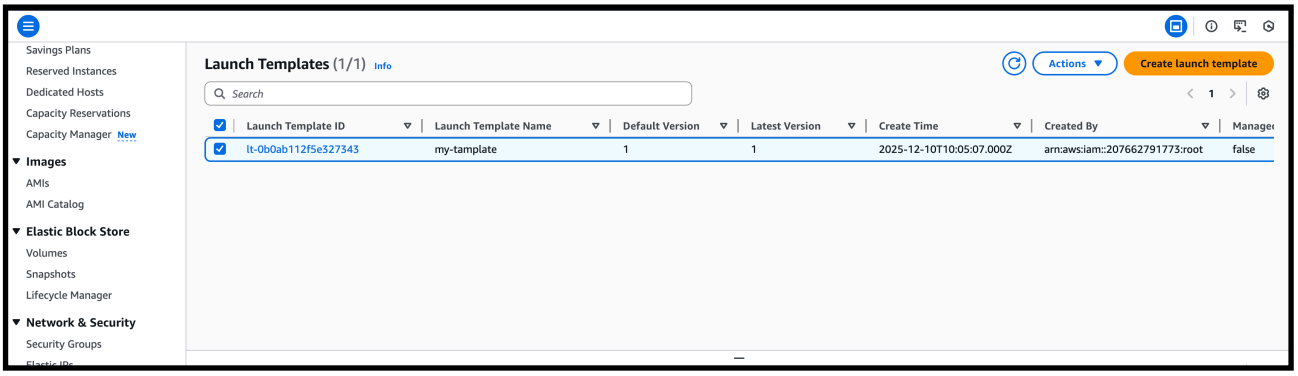
EC2 → Launch Templates → Create launch template

Fill these:



—> Template Name: my-template

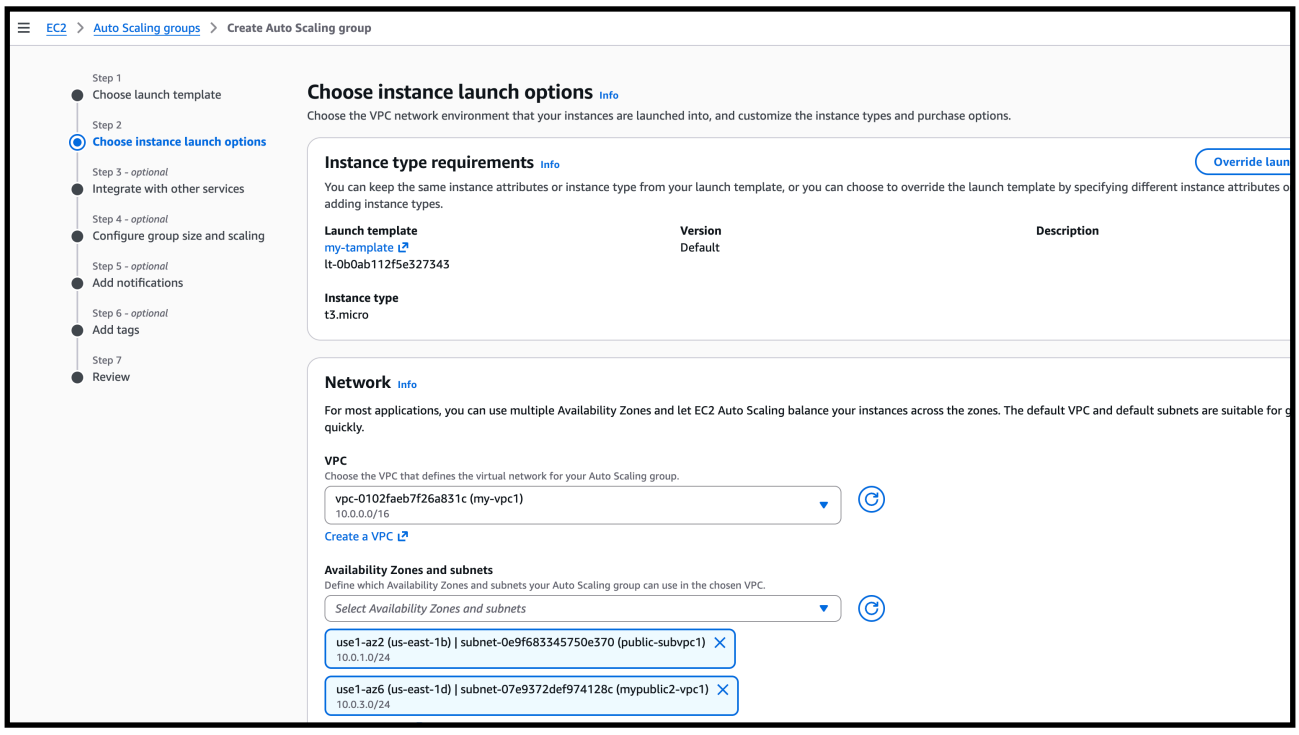
—> select the ami and key pair and SG group
—> and create the launch templete



—> now we have to create a ASG (auto scaling group)
EC2 → Auto Scaling Groups → Create ASG

—> in very first select the launch template which we created

—-> next select the vpc which we created and then select the availability zones of subnets from that vpc



—> click next

—-> after that we have to

Load Balancing (IMPORTANT!)

Select:

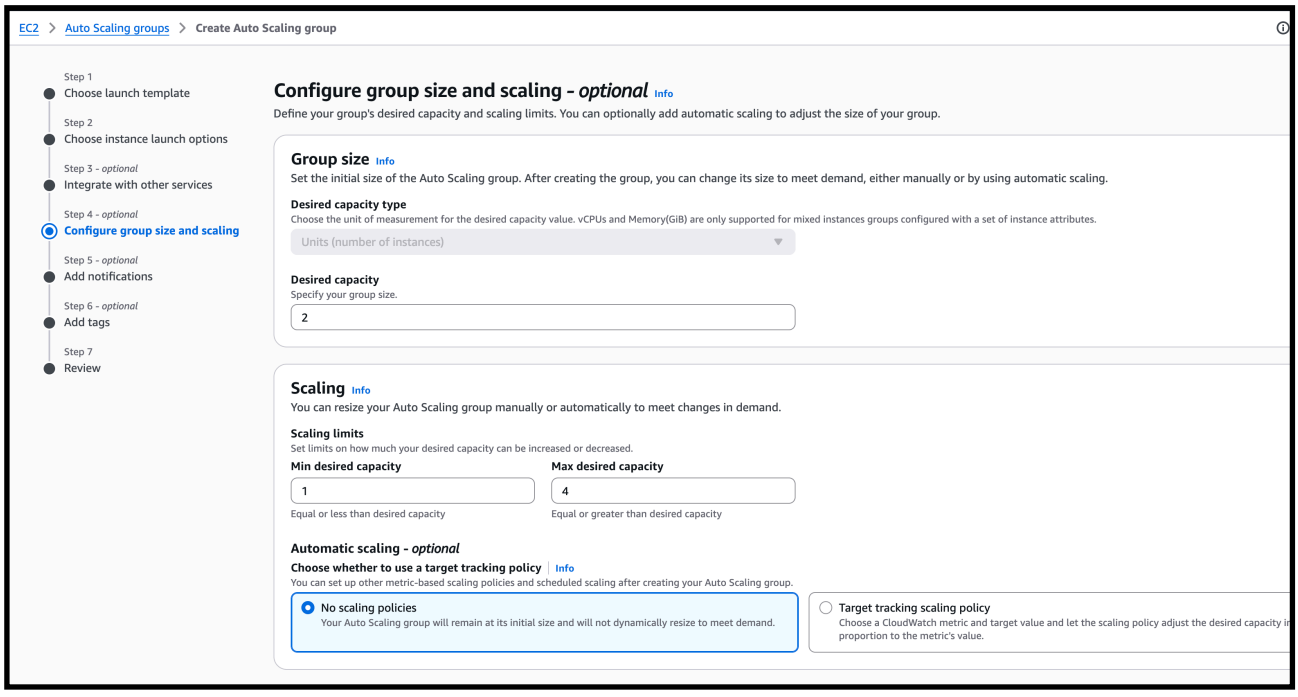
✔ **Attach to an existing load balancer**
Choose the **target group** you created earlier (the one attached to ALB).

Example: my-target-group

—> This ensures new instances automatically register to ALB.

—>. Set Group Size scaling

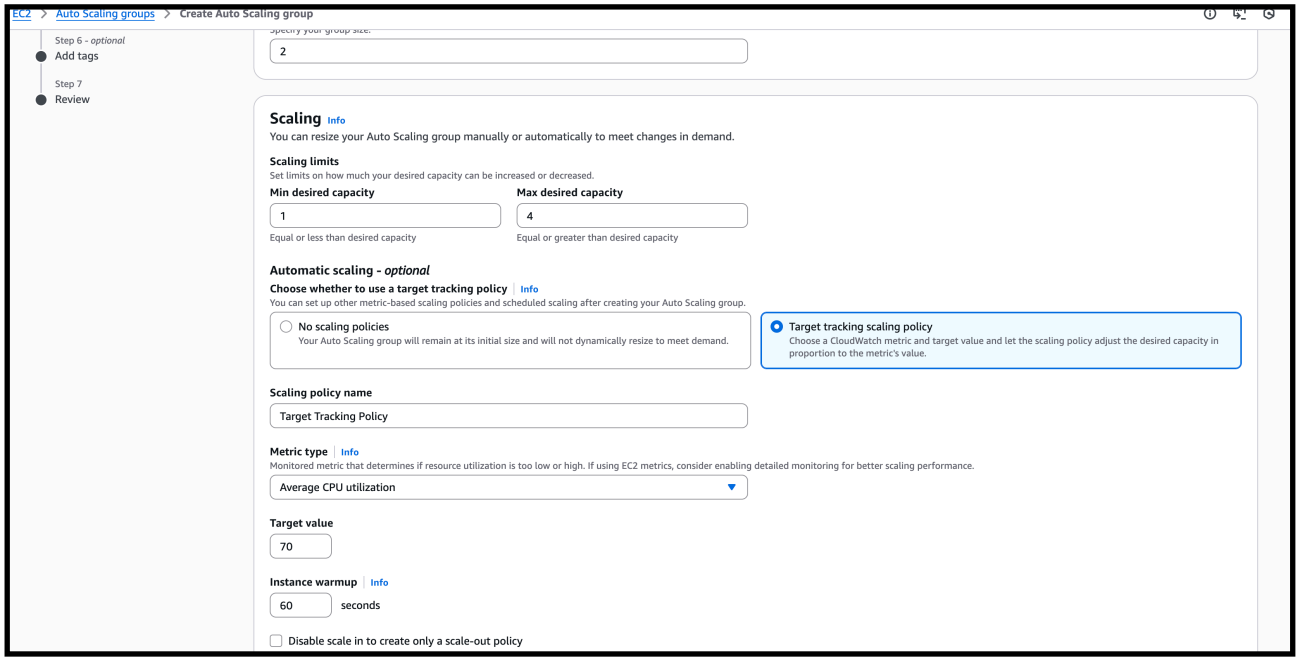
Desired capacity: 2
Minimum capacity: 1
Maximum capacity: 4



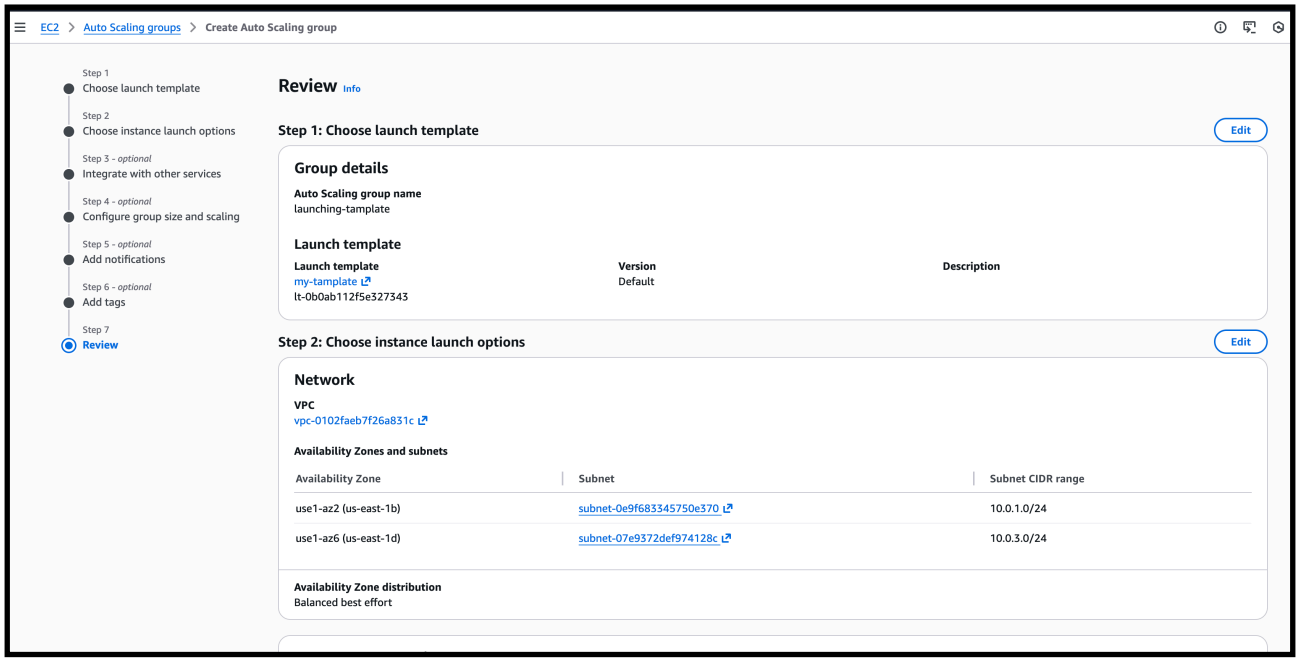
—> automatic scaling
—> select the target tracking scaling policy

Policy Settings

- **Metric type:** Average CPU Utilization
- **Target value:** 70



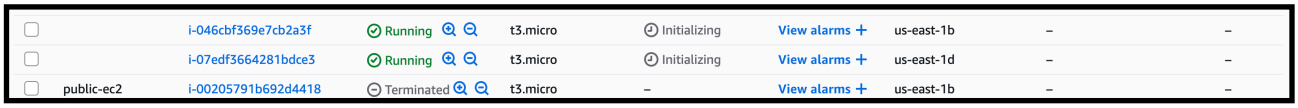
—-> after next review the ASG and create the auto scaling group



- Watch CPU of all EC2 in the group
- If **CPU > 70% for ~2 minutes**
- It **launches a new EC2 instance automatically**
- The new EC2 attaches to:
  ✔ Target Group
  ✔ Load Balancer
  ✔ VPC
  ✔ Security Group

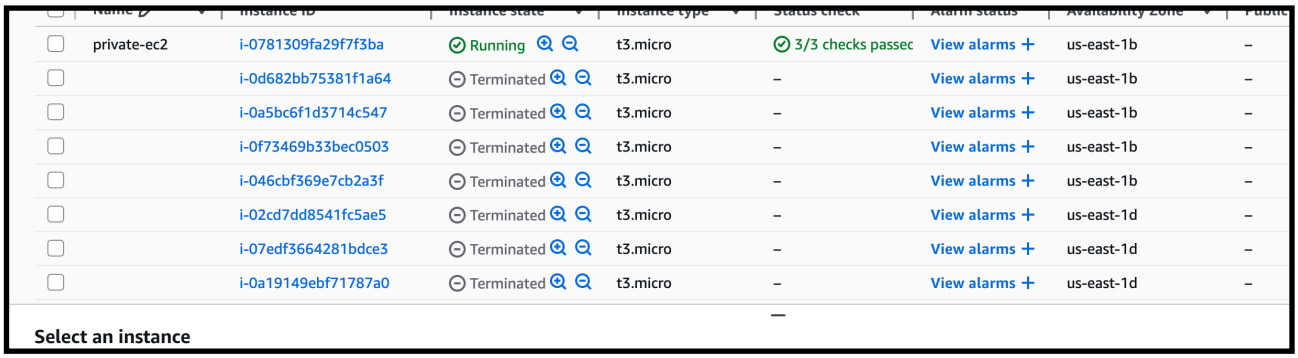
Everything works automatically.

—> check the instances based on cpu it will automatically launch the instance



☐		i-046cbf369e7cb2a3f	Running	t3.micro	Initializing	View alarms +	us-east-1b	–	–
☐		i-07edf3664281bdce3	Running	t3.micro	Initializing	View alarms +	us-east-1d	–	–
☐	public-ec2	i-00205791b692d4418	Terminated	t3.micro	–	View alarms +	us-east-1b	–	–

—> so every thing is working automatically



	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public
☐	private-ec2	i-0781309fa29f7f3ba	Running	t3.micro	3/3 checks passec	View alarms +	us-east-1b	–
☐		i-0d682bb75381f1a64	Terminated	t3.micro	–	View alarms +	us-east-1b	–
☐		i-0a5bc6f1d3714c547	Terminated	t3.micro	–	View alarms +	us-east-1b	–
☐		i-0f73469b33bec0503	Terminated	t3.micro	–	View alarms +	us-east-1b	–
☐		i-046cbf369e7cb2a3f	Terminated	t3.micro	–	View alarms +	us-east-1b	–
☐		i-02cd7dd8541fc5ae5	Terminated	t3.micro	–	View alarms +	us-east-1d	–
☐		i-07edf3664281bdce3	Terminated	t3.micro	–	View alarms +	us-east-1d	–
☐		i-0a19149ebf71787a0	Terminated	t3.micro	–	View alarms +	us-east-1d	–

Select an instance